



Hantro VC9000E JPEG Encoder

API User Manual

Document Revision 1.07

19 September 2024

This document is compatible with the following product:

Hantro VC9000E software release version 2.4.123

VERISILICON

LEVEL B: CONFIDENTIAL - DISTRIBUTION RESTRICTED

Legal Notices

COPYRIGHT INFORMATION

This document contains proprietary information of VeriSilicon Holdings Co., Ltd. VeriSilicon reserves the right to make changes to any products herein at any time without notice. VeriSilicon does not assume any responsibility or liability arising out of the application or use of any product described herein, except as expressly agreed to in writing by VeriSilicon; nor does the purchase or use of a product from VeriSilicon convey a license under any patent rights, copyrights, trademark rights, or any other of the intellectual property rights of VeriSilicon or third parties.

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein may be directly re-distributed or may be adapted for re-distribution by SoC vendors who have licensed the related Hantro core IP, with the understanding that the re-distribution is limited to those customers of the SoC vendor who have active and valid NDA or licenses applicable for the SoC which contains the related Hantro core IP. Otherwise, the information contained herein is not to be used by or disclosed to the third parties without the express written permission of an officer of Hantro Corporation or VeriSilicon Holdings Co., Ltd.

(VeriSilicon Distribution **LEVEL B: Confidential - Distribution Restricted**)

Upon request VeriSilicon provides this document in Word format for adaptive inclusion in documentation intended for the SoC vendor's customers. VeriSilicon requests a pre-release copy of the adaptation for review and approval. In the adapted document please identify the technology and features described in this document using the Hantro trademark and include a reference to the copyright owner, for example: "This section describes the Hantro[®] VPU and contains copyright material disclosed with permission of VeriSilicon Holdings Co., Ltd, who has authorized re-distribution of this material restricted to those NDA partners and licensees of *[the SoC vendor]* who are engineering *[SoC vendor's product]* which includes the discussed Hantro IP."

TRADEMARK ACKNOWLEDGMENT

VeriSilicon, the VeriSilicon logo, Hantro and the Hantro logo are the trademarks of VeriSilicon

Holdings Co., Ltd. in the United States and/or other jurisdictions. All other trademarks are the property of their respective holders.

For our current distributors, sales offices, design resource centers, and product information, visit our web site located at <http://www.verisilicon.com>.

VeriSilicon Confidential, Copyright © 2024 by VeriSilicon Holdings Co., Ltd. All rights reserved worldwide.

Preface

This chapter introduces the *Hantro VC9000E JPEG Encoder API User Manual*.

About This Document

This document introduces the application programming interface (API) of Hantro VC9000E JPEG encoder IP. It describes the data types, enumerations, structures, and functions in the API, which is written in C code. It also provides a programming overview and examples for your reference.

Additional Reading

- Hantro VC9000E Video Encoder Hardware Features
- Hantro VC9000E JPEG Encoder Hardware Features
- Hantro VC9000E Video Encoder Memory Buffer and Format Organization
- Hantro VC9000E JPEG Encoder Test Bench User Manual
- Hantro VC9000E Video Encoder Wrapper Layer API User Manual
- **ISO/IEC International Standard 10918-1:1994.** Information technology. Digital compression and coding of continuous-tone still images.
- **ITU-T Rec. H.265:** High-efficiency video coding
- **ITU-T Rec. H.264:** Advanced video coding for generic audiovisual services
- **ITU-R REC. BT.601:** Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios
- **ITU-R REC. BT.709:** Parameter values for the HDTV standards for production and international programme exchange
- **ITU-R REC. BT.2020:** Parameter values for ultra-high-definition television systems for production and international programme exchange

Table of Contents

Legal Notices	1
Preface	3
1 Overview	12
1.1 IP Introduction	12
1.2 Supported Codec Standards	12
2 Encoder Programming Overview	13
2.1 Initialization	13
2.1.1 Multi-Stream Support	14
2.2 Configuration	14
2.2.1 Thumbnail	14
2.2.2 Encoded Picture Size	15
2.2.3 Frame Size	16
2.2.4 Coding Types.	17
2.3 Pre-Processing.	19
2.3.1 Color Space Conversion	19
2.3.2 Rotation.	20
2.3.3 Cropping	21

2.4	Encoding	21
2.5	Quantization Level and Output Size	22
2.6	Release	24
3	JPEG Encoder Features	25
3.1	Encoding Formats	25
3.2	ROI Map	25
3.3	Pre-Processing	26
3.3.1	Color Space Conversion	26
3.3.2	On-the-Fly OSD Mixer	27
3.4	Stream Segment for Low Latency	27
3.5	Osdmap	28
4	Video Frame Storage Format	30
5	Application Example	32
6	Module Documentation	34
6.1	Numeric Data Types	34
6.2	JPEG Encoder API	35
6.2.1	Marco Definition	35
6.2.1.1	JPEGENC_STREAM_MIN_BUF0_SIZE	35
6.2.2	Enumeration Type	35
6.2.2.1	JpegEncRet	35
6.2.2.2	JpegEncFrameType	37
6.2.2.3	JpegEncPictureScanType	39

6.2.2.4	JpegEncPictureRotation	40
6.2.2.5	JpegEncColorConversionType	40
6.2.2.6	JpegEncCodingType	41
6.2.2.7	JpegEncCodingMode	41
6.2.2.8	JpegEncAppUnitsType	42
6.2.2.9	JpegEncTableMarkerType	42
6.2.2.10	JpegEncThumbFormat	43
6.2.2.11	JpegEncRcMode	43
6.2.2.12	JpegEncParamType	43
6.2.3	Functions	44
6.2.3.1	JpegEncInit()	44
6.2.3.2	JpegEncRelease()	45
6.2.3.3	JpegEncSetParams()	45
6.2.3.4	JpegEncGetParams()	46
6.2.3.5	JpegEncSetPictureSize()	46
6.2.3.6	JpegEncSetThumbnail()	47
6.2.3.7	JpegEncEncode()	47
6.2.3.8	JpegEncEncodeRun()	48
6.2.3.9	JpegEncEncodeWait()	48

6.3	Helper Functions	49
6.3.1	Functions	49
6.3.1.1	JpegEncGetApiVersion()	49
6.3.1.2	JpegEncGetBuild()	49
6.3.1.3	JpegEncCoreHasFeatures()	50
6.3.1.4	JpegEncGetBitsPerPixel()	50
6.3.1.5	JpegEncGetAlignedStride()	51
6.3.1.6	JpegGetLumaSize()	51
6.3.1.7	JpegSetLumaSize()	52
6.3.1.8	JpegGetChromaSize()	52
6.3.1.9	JpegSetChromaSize()	53
6.3.1.10	JpegGetQpHdr()	53
6.3.1.11	JpegEncGetEwl()	54
6.3.1.12	JpegEncGetOverlaySlice()	54
6.3.1.13	JpegEncGetPerformance()	55
6.3.1.14	JpegGetSbiSupport()	55
6.4	Low-Latency Control API	56
6.4.1	Functions	56
6.4.1.1	JpegEncGetEncodedMbLines()	56
6.4.1.2	JpegEncSetInputMBLines()	56
6.5	MJPEG Bit Rate Control API	57
6.5.1	Functions	57
6.5.1.1	JpegEncSetRateCtrl()	57
6.5.1.2	JpegEncGetRateCtrl()	58

6.6	Log System API	58
6.6.1	Marco Definition	60
6.6.1.1	APITRACEERR	60
6.6.1.2	APITRACEWRN	60
6.6.1.3	APITRACE_ERR	61
6.6.1.4	CMLTRACE	61
6.6.1.5	RC_LOG_E	61
6.6.2	Enumeration Type	61
6.6.2.1	VCEncLogEnvControl	61
6.6.2.2	VCEncLogOutput	62
6.6.2.3	VCEncLogFileType	62
6.6.2.4	VCEncLogTraceType	63
6.6.2.5	VCEncLogCheckType	63
6.6.2.6	VCEncLogLevel	64
6.6.3	Functions	64
6.6.3.1	VCEncLogInit()	64
6.6.3.2	VCEncTraceMsg()	65
6.6.3.3	VCEncCheckMsg()	66
6.6.3.4	VCEncLogDestory()	66
6.6.3.5	EncTraceRegs()	66
6.6.3.6	VCEncLogGetEnvSetting()	67
7	Data Structure Documentation	68

7.1	jpeg_ufbc_afbc_param	68
7.2	jpeg_ufbc_param	68
7.3	jpeg_ufbc_pvrlic_param	69
7.4	JpegEncCfg	69
7.5	JpegEncColorConversion	77
7.6	JpegEncIn	78
7.6.1	Field Documentation	80
7.6.1.1	dec400Enable	80
7.7	JpegEncOut	81
7.8	JpegEncQuality	82
7.8.1	Field Documentation	82
7.8.1.1	factor	82
7.9	JpegEncRateCtrl	82
7.10	JpegEncThumb	84
7.11	VCEncLogSetting	84
Appendix: Glossary		85
Document Revision History		88

List of Figures

2.1 JPEG Encoding Process	13
2.2 Encoded Picture Width and Horizontal Stride Example	16
2.3 Whole Frame Coding and Sliced Frame Coding	18
2.4 Encoded Picture Size after Rotation	20
2.5 Relation Between Quantization Levels and JFIF Output Size	23
3.1 Stream Segment Buffer Parameters	28

List of Tables

2.1 JFIF Output Size for Different Resolutions with Different Quantization Levels . . .	23
4.1 Color Component Size of Different Input Formats	30

1 Overview

This chapter gives an overview of the IP core.

1.1 IP Introduction

The multi-format VC9000E video encoder is a single-core solution that supports encoding streams with a picture size up to 32768 x 32768. It features small silicon footprint, low power consumption, and negligible load on the host CPU.

1.2 Supported Codec Standards

The APIs discussed in this version of the document are compatible with the following video codec standards:

- JPEG ITU-T Rec. T.81 (09/92), baseline
- JFIF JPEG File Interchange Format, see <http://www.w3.org/Graphics/JPEG/jfif3.pdf>

2 Encoder Programming Overview

Figure 2.1 shows the VC9000E JPEG encoding process, which includes initialization, configuration, pre-processing, encoding, and release.

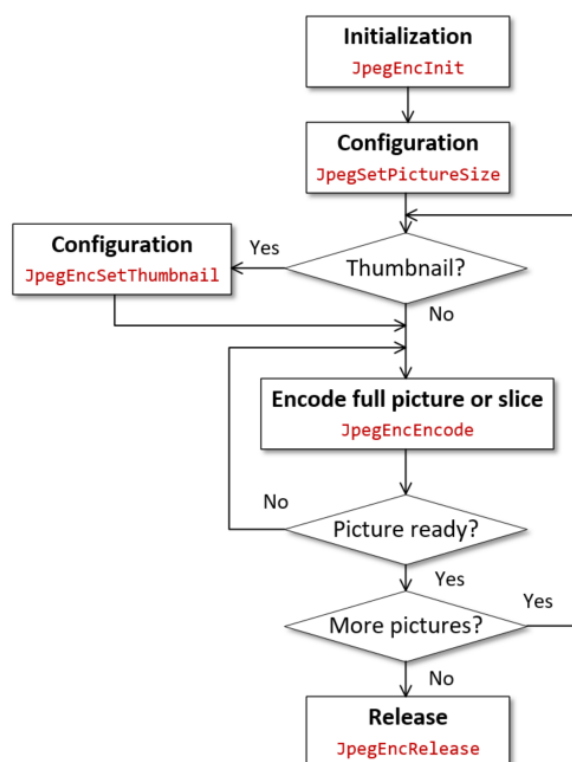


Figure 2.1 JPEG Encoding Process

2.1 Initialization

The encoder before being used must be initialized by calling `JpegEncInit()`. This call allocates the required resources and executes setup steps. If it succeeds, the call returns a new encoder instance, which is used as an identifier for subsequent operations.

If the initialization is successful, `JPEGENC_OK` is returned. Otherwise, an error code is returned to indicate the exceptions during initialization.

Certain parameters must be set and cannot be changed during encoding.

2.1.1 Multi-Stream Support

The encoder supports multiple software instances sharing a single piece of encoder hardware. These instances can be initialized to encode different streams. This cannot be done at the same time, because the hardware encodes only one frame at a time.

An application can adopt the following methods to manage multiple streams:

1. The application maintains and manages multiple encoder instances.
After encoding one frame, the application switches to another in either the same instance or a different instance.
2. The application utilizes multiple processes, with each process managing one stream.
The encoder driver protects hardware resources to avoid conflicts between different processes. Once a frame is encoded, the hardware resource is released and can be used by other processes.

2.2 Configuration

After the encoder is initialized, encoding parameters can be configured as required.

2.2.1 Thumbnail

An external thumbnail image is specified by using the `JpegEncSetThumbnail()` function. Formats compliant with JFIF 1.02 (JPEG, 8-bit) and 24-bit RGB formats are supported. The thumbnail data must be prepared externally and provided to the encoder in a separate buffer from the input picture buffer. Thumbnail image is inserted into the final JPEG file by software.

2.2.2 Encoded Picture Size

The size of an encoded picture is specified during configuration. The size of an input picture captured by a camera may differ from the size of the output encoded picture. The original size of an input picture may be larger than the size of the output encoded picture. Moreover, the output encoded picture can be part of the input picture. For more information about cropping, see Section *Cropping*.

The input and output picture sizes must meet the following requirements:

- The maximum input picture size is 32768 x 32768, which may vary with hardware configuration.
- The encoded picture width must be a multiple of 2.
- The encoded picture height must be a multiple of 2.
- The smallest encoded picture size is 32 x 32, if rotation, is 64x64.
- The maximum encoded picture size is 32768 x 32768, which may vary with hardware configuration.

Encoded Picture Width Limitation

The horizontal stride of an input picture must be a multiple of 2. That means the memory offset from the start of a pixel row to the beginning of the next pixel row is always a multiple of 2. This horizontal stride in a multiple of 16 is recommended for an input picture to enable hardware to efficiently fetch the input picture. This is reflected in the initial value of the configuration parameters.

The encoded picture height must be a multiple of 2.

Figure 2.2 shows an example of the encoded picture width and horizontal stride.

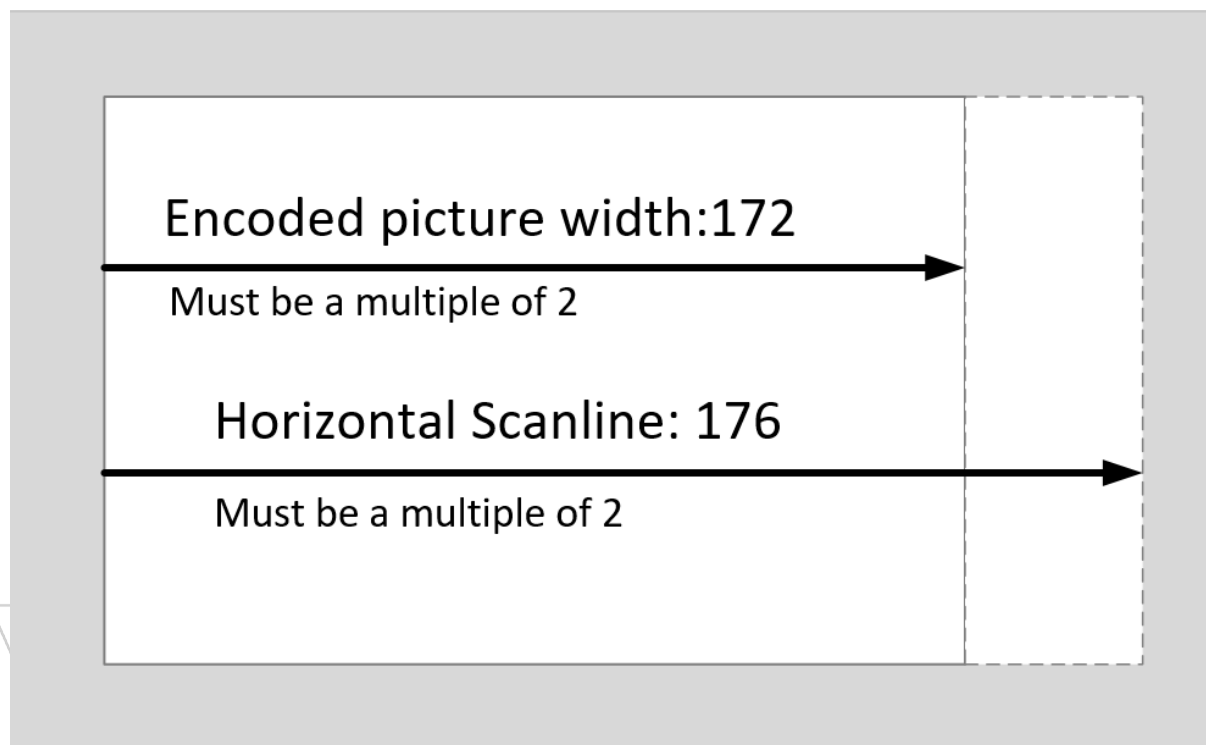


Figure 2.2 Encoded Picture Width and Horizontal Stride Example

The encoded picture size can be changed after initialization by calling `JpegEncSetPictureSize()`. If the encoded picture size the size of the rotated input picture, the size that is set during initialization is the size of the rotated picture.

2.2.3 Frame Size

Pictures in the YUV420 format have the following limitations on the input picture size and the encoded picture size:

- $\text{encoded_width} \% 2 = 0$
- $\text{encoded_height} \% 2 = 0$
- $\text{input_width} \geq \text{encoded_width}$
- $\text{input_width} \% 2 = 0$
- $\text{input_height} \geq \text{encoded_height}$

- $\text{input_height} \% 2 = 0$

Pictures in the YUV422 and RGB formats have the following limitations on the input picture size and the encoded picture size:

- $\text{encoded_width} \% 2 = 0$
- $\text{encoded_height} \% 2 = 0$
- $\text{input_width} \geq \text{encoded_width}$
- $\text{input_width} \% 2 = 0$
- $\text{input_height} \geq \text{encoded_height}$
- $\text{input_height} \% 2 = 0$

where:

• $\%$: remainder operator

• $\&$: bitwise AND operator

• $\sim$: bitwise NOT operator

See also

`JpegEncCfg.inputWidth`

`JpegEncCfg.inputHeight`

`JpegEncCfg.codingWidth`

`JpegEncCfg.codingHeight`

2.2.4 Coding Types

Different frame coding types can be selected for the purpose of full-resolution encoding. A single frame can be encoded either as a whole frame or in several slices. In the latter case, the width of each slice is the width of the original picture and the height of each slice is a multiple of 16 for YUV420 baseline encoding, or 8 for YUV422 baseline encoding. The slice height is defined in

MCU rows with a restart interval. Sliced frame coding saves memory and allows simultaneous operation of a camera module and the encoder. **Figure 2.3** shows the two coding types.



Figure 2.3 Whole Frame Coding and Sliced Frame Coding

To encode a whole frame at a time, the steps are as follows:

1. Get the frame.
2. Encode the frame.

To encode one slice of a frame at a time, the steps are as follows:

1. Get the first slice.
2. Encode the first slice.
3. Get the second slice.
4. Encode the second slice.

5. Get the third slice.
6. Encode the third slice.

For whole frame encoding, the whole frame must be retrieved and stored in the memory before encoding.

For sliced frame encoding, each slice is retrieved and encoded separately. Encoding of different slices can be performed at the same time with multiple hardware cores. If all the preceding steps are performed sequentially, each of these slices can use a single slice buffer. Software writes the encoded slice data into one file in order.

See also

`JpegEncCfg.codingType`
`JpegEncCfg.restartInterval`

2.3 Pre-Processing

The VC9000E encoder includes the following pre-processing modules for JPEG encoding:

- Color space conversion
- Rotation
- Cropping

By default, the encoder input is in planar YUV420 format, and cropping and rotation are disabled.

2.3.1 Color Space Conversion

The color space conversion (CSC) module converts RGB formats into planar YUV420 format by vertically sub-sampling chrominance samples and rearranging the samples. The supported input picture formats can be 30-, 24-, 16-, 15- or 12-bit RGB format. This RGB-to-YUV conversion is configured during the encoder configuration by setting `JpegEncCfg.colorConversion` and performed on every input picture.

Note

The YUV422-to-YUV420 conversion is executed by averaging pixels value. This conversion function is built in the module after the RGB-to-YUV module.

See also

`JpegEncCfg.colorConversion`

2.3.2 Rotation

The rotation module rotates a picture by 90 degrees clockwise or counter-clockwise. This function is configured during initialization and takes effect on every encoded picture. Figure 2.4 shows the encoded picture size after rotation.

Note

Picture rotation is supported only in whole frame encoding mode.

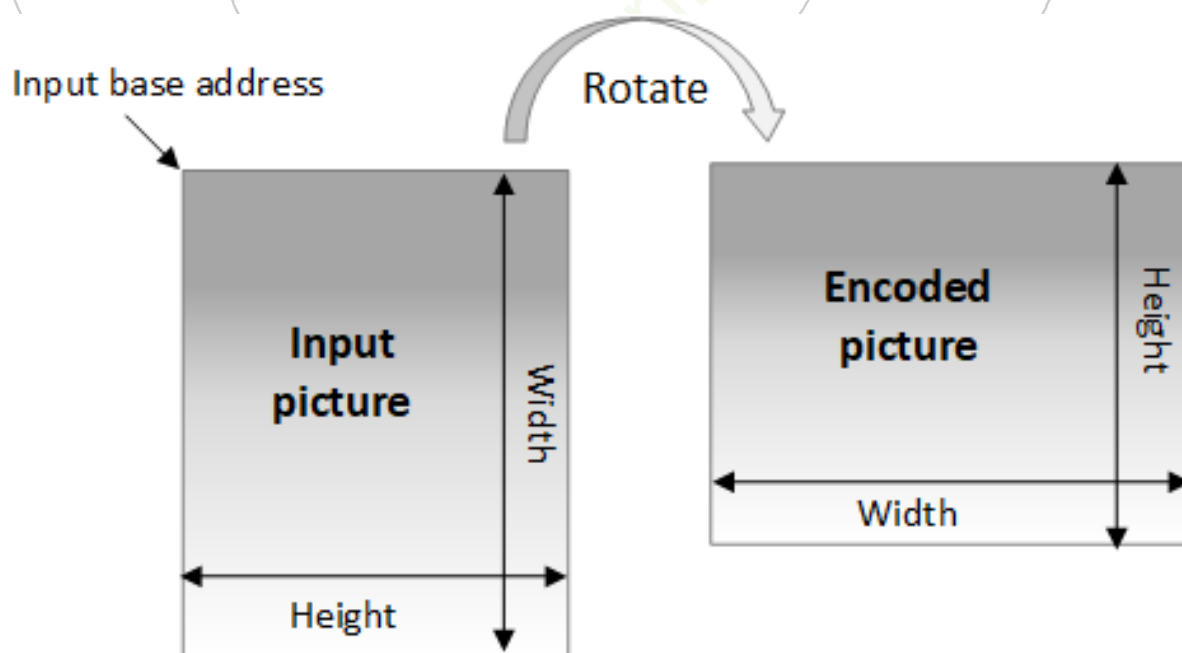


Figure 2.4 Encoded Picture Size after Rotation

See also

`JpegEncCfg.rotation`

2.3.3 Cropping

The picture cropping module crops a small selected portion from a large picture. To configure this function, you need to set the size of the input picture and the top-left corner coordinates of the encoded picture relative to the top-left corner of the input picture.

To enable the cropping module, the following conditions must be satisfied:

- $\text{inputWidth} \geq (\text{codingWidth} + 1) \& (\sim 1)$
- $\text{inputWidth} - \text{xOffset} - \text{codingWidth} \geq 0$
- $\text{inputHeight} - \text{yOffset} - \text{codingHeight} \geq 0$
- The input picture horizontal stride (in pixels) must be a multiple of 2.
- The input picture height must be a multiple of 2.

See also

`JpegEncCfg.inputWidth`
`JpegEncCfg.inputHeight`
`JpegEncCfg.xOffset`
`JpegEncCfg.yOffset`
`JpegEncCfg.codingWidth`
`JpegEncCfg.codingHeight`

2.4 Encoding

After initialization and optional configuration, the encoder is ready to produce a JPEG/JFIF compliant stream.

The supported input formats include planar YUV420, semi-planar YUV420, interleaved YUV422, and 32- and 16-bit RGB, which are specified during initialization. The memory where the buffers are located must be 16-bit aligned, linear, and hardware-accessible. Meanwhile, the addresses sent to the `JpegEncEncode()` function are bus addresses, which are conveyed through hardware registers. The hardware reads the input data based on these addresses.

The output buffer must be specified for both software and hardware. A virtual address pointer specifies the buffer for software access. A bus address specifies the buffer for hardware access. The output buffer size are related to the picture size and quantization level. If the encoder reaches the end of the output buffer, the current frame encoding cannot continue and the encoding process stops. In this case, `JpegEncEncode()` returns `JPEGENC_OUTPUT_BUFFER_OVERFLOW`. If the sliced frame encoding mode is used, `JpegEncEncode()` encodes one slice at a time and returns `JPEGENC_RESTART_INTERVAL`.

Note

If an MMU is configured in the encoder subsystem, the output buffer can not be continuous and the bus address should be the virtual address of IOMMU.

A JPEG picture that contains a thumbnail is encoded in the following steps:

1. Prepare external thumbnail picture data.
2. Call `JpegEncSetThumbnail()` to set up the thumbnail.
3. Allocate an output buffer that is large enough to suit the encoded picture and the thumbnail.
4. Call `JpegEncSetPictureSize()` to configure the picture size and pre-processing functions.
5. Call `JpegEncEncode()` to encode the picture.

After `JPEGENC_FRAME_READY` is returned, the JPEG picture is ready for output.

2.5 Quantization Level and Output Size

An application can define several quantization levels (see `JpegEncInit()`) for use in encoding. A low quantization level leads to low picture quality and small JFIF size. A high quantization level leads to high picture quality and large JFIF size. Table 2.1 lists the output size for different resolutions with different quantization levels.

Table 2.1 JFIF Output Size for Different Resolutions with Different Quantization Levels

Quantization Level	CIF 352x288 [KB]	VGA 640x480 [KB]	1 Mpixel 1280x864 [KB]	2 Mpixel 1600x1232 [KB]	5 Mpixel 2560x1968 [KB]
0	4	9	26	42	97
1	6	13	34	52	115
2	7	16	42	62	134
3	9	19	47	70	150
4	10	21	54	78	166
5	12	25	62	88	186
6	13	29	70	105	229
7	17	38	92	145	326
8	24	53	158	258	595
9	42	94	561	823	1665

Figure 2.6 shows the relation between quantization levels and output size for different resolutions.

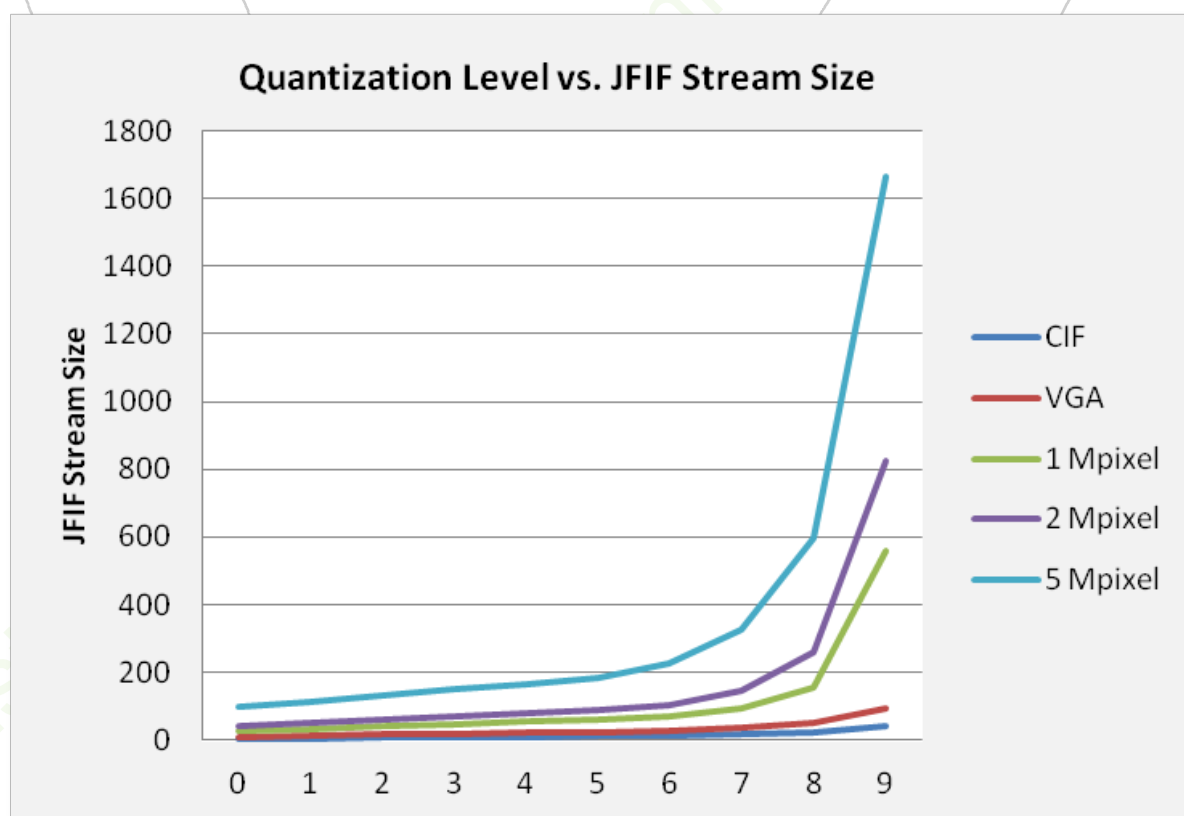


Figure 2.5 Relation Between Quantization Levels and JFIF Output Size

See also`JpegEncCfg.qLevel``JpegEncCfg.qTableLuma``JpegEncCfg.qTableChroma`**2.6 Release**

After encoding is completed, the encoder must be released by using `JpegEncRelease()` to release all allocated resources. If the release fails, the cause may be corrupted software integration of encoder instance or the fact that certain rules are not followed.

3 JPEG Encoder Features

This chapter describes the features of the VC9000E JPEG encoder and related data structures for enabling these features.

3.1 Encoding Formats

The VC9000E JPEG encoder supports YUV420, YUV422, and monochrome JPEG encoding formats.

`JpegEncCfg.codingMode` is used to specify the encoding format. If an input format is not supported, the pre-processing module converts the input format to the required format.

3.2 ROI Map

ROI map is an accessible buffer through its physical address. You should fill in this buffer before encoding with the ROI feature. The ROI map defines a region of interest (ROI) where the picture quality out of this region degrades after encoding. The ROI location is related to the effective encoding area instead of the input picture area. Every bit in the ROI map buffer indicates whether an MCU is ROI or non-ROI. Value "1" indicates the MCU is encoded as an ROI region and value "0" indicates the MCU is encoded as a non-ROI region. Before encoding a non-ROI MCU, a filter is used to degrade the quality of the MCU with fewer bits.

The ROI map feature is configurable by using the following parameters:

- `JpegEncCfg.enableRoimap` indicates whether to enable the ROI map feature.

- `JpegEncCfg.nonRoiLevel` specifies the level of the pre-defined filters for the non-ROI region.

Its value ranges from 0 to 9. If you specify the value as 10, you should define your own filter coefficients for the non-ROI region. The coefficients should be filled in array `JpegEncCfg.filter[]`.

- `JpegEncCfg.filter` specifies the coefficients of the filter specified for the non-ROI region.

There are 128 coefficients, where values in 0 to 63 coefficients are used for luma, and values in 64 to 127 coefficients are used for chroma.

After an ROI map is configured, you can specify the ROI map buffer by using `JpegEncIn.busRoiMap` when `JpegEncEncode()` is called.

3.3 Pre-Processing

3.3.1 Color Space Conversion

Color space conversion is supported only when the value of `EWLHwConfig_t.rgbEnable` is 1.

The input RGB formats are converted into YUV formats by the following equation:

$$\begin{aligned}
 Y' &= A \times r + B \times g + C \times b \\
 Y &= \text{clip}(Y' + \text{LumaOffset}) \\
 U &= \text{clip}(E \times b - G \times Y' + \text{Const}) \\
 V &= \text{clip}(F \times r - H \times Y' + \text{Const})
 \end{aligned}$$

Where,

- A, B, C, D, E, F, G, H, and LumaOffset are the parameters in `JpegEncColorConversion`.
- clip() clips the calculation result into the range of corresponding bit depth.
- Const is the constant offset for chroma, which is 128 for 8 bits.

When `JpegEncColorConversion.type` is set to `JPEGENC_RGBTOYUV_USER_DEFINED`, color conversion parameters can be customized according to the preceding equation. Some predefined parameters are used to define the attributes of the input RGB picture, such as BT601 and BT709. For more information, see Section *JpegEncColorConversion*.

Note

LumaOffset is supported only when the value of `EWLHwConfig_t.cscExtendSupport` is 1.

3.3.2 On-the-Fly OSD Mixer

The encoder supports displaying a rectangular picture combined with the input picture. The generated combined picture is transferred to the encoder as the final input.

This feature is only available when the value of `EWLHwConfig_t.OSDSupport` is 1.

See also

`JpegEncCfg.olEnable` [MAX_OVERLAY_NUM]

`JpegEncCfg.olFormat` [MAX_OVERLAY_NUM]

3.4 Stream Segment for Low Latency

Stream segment is supported only when `EWLHwConfig_t.encStreamSegmentSupport` is non-zero.

In low latency cases, encoded streams should be transferred as soon as possible. The output stream buffers can be grouped into fixed-size segment buffer. After a segment buffer is filled, an interrupt is generated to instruct you to process the segment buffer.

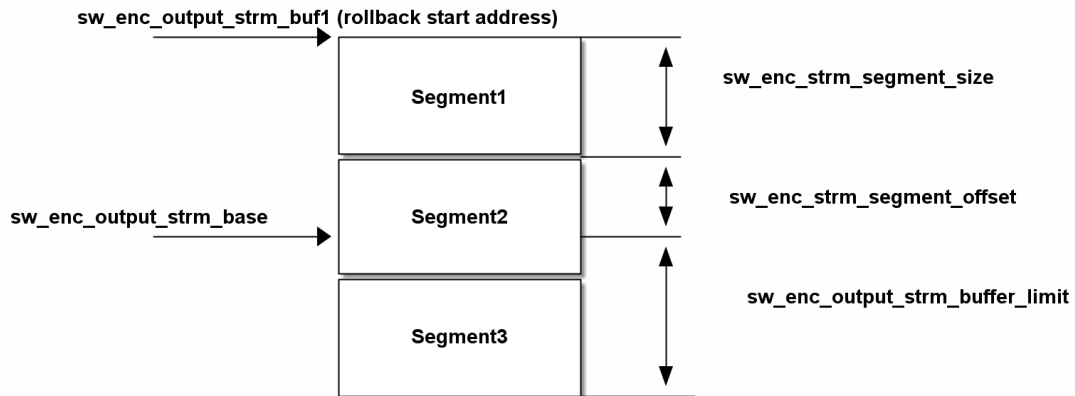


Figure 3.1 Stream Segment Buffer Parameters

See also

`JpegEncCfg.streamMultiSegmentMode`
`JpegEncCfg.streamMultiSegmentAmount`
`JpegEncCfg.streamMultiSegmentSize`
`JpegEncCfg.streamMultiSegCbFunc`
`JpegEncCfg.streamMultiSegCbData`

3.5 Osdmap

OSD Map is a buffer specified by the user, where every 4 bits correspond to a pixel's color and alpha value (up to 12 colors and alpha values). Then the user can mix the specified area with alpha and the original image according to the specified color in the Map Buffer, and the mixed result is used as the input for encoding. This function can be used to scramble any area. For example, in car applications, color blocks can be applied to the facial part. In the past, the mosaic function could only support up to 12 regions of mosaic in VCE, and using OSD Map did not have this limitation.

The Data in OSD Map buffer are defined as following:

- Each 8x8 block corresponds to a 4-bit control information. The value of these 4-bits determines the color on this 8x8 stack.

- The MSB of one byte defines the block before the block defined in the LSB.

The specific 4-bit control information encoding is as follows:

- 4'd0: Do not stack any information
- 4'd1: Overlay the first color and alpha configuration
- 4'd2: Overlay the second color and alpha configuration
- 4'd8: Overlay the eighth color and alpha configuration
- 4'd12: Overlay the twelfth color and alpha configuration
- 4'd13~4'd15: Invalid

Note:

- Comparing with overlay and mosaic, OSDMap has the lowest priority.
- OSDMap and overlay supertile should not be both turned on at the same time.
- The sum of available colors in the OSD area and OSDMap cannot be greater than 12, because OSDMAP and OSD share registers to represent the corresponding colors in the Map.
- It is not recommended to open both OSDMAP and Mosaic functions simultaneously, because the goals of OSDMAP and Mosaic functions are similar.
- You can get information on whether this feature is supported from `EWLHwConfig_1.OSDMapSupport`. When this value is 1, it indicates support.

To use this feature, you need to configure the video encoder as follows:

- you can set the `JpegEncCfg.osdMapEnable` to 1 to enable osdmap feature.
- Set `JpegEncCfg.osdMapStride` to OSD Map stride in byte. OSDMap 4bits perform 8x8 or 4x4 block, so `osdMapStride` should not less than $(codingWidth + 15) >> 4$ or $(codingWidth + 7) >> 3$.
- Set `JpegEncCfg.osdMapBlockSize` to OSD Map block size. 8 means 8x8, 4 means 4x4.
- Set `JpegEncCfg.osdMapAlpha[i]` to the alpha value for color `i`.
- Set `JpegEncCfg.osdMapY[i]` to the OSD Map format color[i] Y value(0..255).
- Set `JpegEncCfg.osdMapU[i]` to the OSD Map format color[i] U value(0..255).
- Set `JpegEncCfg.osdMapV[i]` to the OSD Map format color[i] V value(0..255).

4 Video Frame Storage Format

The VC9000E JPEG encoder supports the following input picture formats:

- Planar YUV420
- Semi-planar YUV420
- Interleaved YUV422
- 32-bit RGB
- 16-bit RGB

10-bit color formats are not supported for JPEG encoder.

When the encoder software is compiled, the data endianness of the input picture is set and cannot be changed during run-time. The component size of each color format depends on the input picture size, as listed in the following table.

Table 4.1 Color Component Size of Different Input Formats

Input Formats	Y (Bytes)	U (Bytes)	V (Bytes)	U+V (Bytes)	R+G+B (Bytes)
Planar YUV420	$W \times H$	$(W/2) \times (H/2)$	$(W/2) \times (H/2)$	-	-
Semi-planar YUV420	$W \times H$	-	-	$(W \times H) / 2$	-
16-bit RGB	-	-	-	-	$W \times H \times 2$
32-bit RGB	-	-	-	-	$W \times H \times 4$

where:

- W = input picture width

- H = input picture height

See also

`JpegEncCfg.frameType`



5 Application Example

This chapter provides a code sample of encoding a VGA picture, which includes the API functions and external functions (OS-specific) `ReadFrame()`, `SaveStream()` and `AllocateOutputBuffer()`. This code sample has been simplified without error handling and does not guarantee successful compilation.

```
#include "JpegEncApi.h"
#include <stdlib.h>

JpegEncRet ret;
JpegEncInst encoder;
JpegEncCfg cfg;
JpegEncIn encIn;
JpegEncOut encOut;
/* Output buffer size: For most images 1 byte/pixel is enough
   for high quality JPEG */
u32 outbuf_size = 640*480;
u32 pict_bus_address = 0;
u32 *outbuf_virt_address = NULL;
u32 output_bus_address = 0;
/* Step 1: Initialize an encoder instance */
cfg.qLevel = 5;
cfg.frameType = JPEGENC_YUV420_PLANAR;
cfg.markerType = JPEGENC_SINGLE_MARKER;
cfg.unitsType = JPEGENC_DOTS_PER_INCH;
cfg.xDensity = 72;
cfg.yDensity = 72;
if((ret = JpegEncInit(&cfg, &encoder)) != JPEGENC_OK)
{
    /* Handle the error situation */
    goto end;
}
/* Step 2: Configuration of picture size */
/* VGA resolution, no cropping */
cfg.inputWidth = 640;
cfg.codingWidth = 640;
cfg.inputHeight = 480;
cfg.codingHeight = 480;
cfg.xOffset = 0;
cfg.yOffset = 0;
cfg.rotation = JPEGENC_ROTATE_0;
cfg.codingType = JPEGENC_WHOLE_FRAME;
/* Restart interval 5 MCU rows (= 80 pixel rows), average quantization,
   encode whole frame at once */
cfg.restartInterval = 5;
if((ret = JpegEncSetPictureSize(encoder, &cfg)) != JPEGENC_OK)
{
    /* Handle the error situation */
    goto close;
}
/* Step 3: Allocate linear output buffer */
AllocateOutputBuffer(&output_bus_address, &outbuf_virt_address,
```

```

        outbuf_size);
encIn.pOutBuf      = outbuf_virt_address;
encIn.busOutBuf    = output_bus_address;
encIn.outBufSize   = outbuf_size; /* bytes */
/* Step 4: Get the picture and setup input */
/* Allocate linear picture memory and read the picture */
ReadFrame(&pict_bus_address);
/* Enable all frame headers */
encIn.frameHeader = 1;
/* We use one linear buffer for the input image: YYYY...UU...VV... */
encIn.busLum = pict_bus_address;
encIn.busCb  = encIn.busLum + cfg.inputWidth * cfg.inputHeight;
encIn.busCr  = encIn.busCb + (cfg.inputWidth/2) * (cfg.inputHeight/2);
/* Step 5: Encode the picture */
/* Loop until the frame is ready */
do
{
    ret = JpegEncEncode(encoder, &encIn, &encOut);
    switch (ret)
    {
        case JPEGENC_RESTART_INTERVAL:
        case JPEGENC_FRAME_READY:
            SaveStream(encIn.pOutBuf, encOut.jfifSize);
            break;
        default:
            /* All the others are error codes */
            break;
    }
}
while (ret == JPEGENC_RESTART_INTERVAL);
close:
/* Last Step: Release the encoder instance */
if((ret = JpegEncRelease(encoder)) != JPEGENC_OK)
{
    /*
     * There is not much to do for an error here,
     * just a notification message. There has to be a fault
     * somewhere else that caused a failure of the
     * encoder release
     */
}
/* release output buffer and picture buffer, if needed */
end: /* nothing else to do */

```

6 Module Documentation

6.1 Numeric Data Types

This section provides the definitions of numeric data types.

Typedef

- typedef int8_t **i8**
Signed 8-bit integer.
- typedef uint8_t **u8**
Unsigned 8-bit integer.
- typedef int16_t **i16**
Signed 16-bit integer.
- typedef uint16_t **u16**
Unsigned 16-bit integer.
- typedef int32_t **i32**
Signed 32-bit integer.
- typedef uint32_t **u32**
Unsigned 32-bit integer.
- typedef int64_t **i64**
Signed 64-bit integer.
- typedef uint64_t **u64**
Unsigned 64-bit integer.
- typedef **u64 ptr_t**
Buffer address.

6.2 JPEG Encoder API

Marco Definition

- `#define JPEGENC_STREAM_MIN_BUF0_SIZE 1024`
- `#define DEC400_HEADER_BUF_SIZE (128)`

The size of DEC400 Tile Status Header Data if existing.

Typedef

- `typedef const void * JpegEncInst`

JPEG encoder instance.

6.2.1 Marco Definition

6.2.1.1 JPEGENC_STREAM_MIN_BUF0_SIZE

```
#define JPEGENC_STREAM_MIN_BUF0_SIZE 1024
```

define the minimum size of the buffer which is used to store JPEG headers generated by software

6.2.2 Enumeration Type

6.2.2.1 JpegEncRet

```
enum JpegEncRet
```

Function return values.

Enumerator

JPEGENC_FRAME_READY	Frame ready
JPEGENC_RESTART_INTERVAL	Restart interval ready
JPEGENC_OK	Operation succeeds
JPEGENC_ERROR	Error
JPEGENC_NULL_ARGUMENT	Null argument
JPEGENC_INVALID_ARGUMENT	Invalid argument
JPEGENC_MEMORY_ERROR	Memory error
JPEGENC_INVALID_STATUS	Invalid status
JPEGENC_OUTPUT_BUFFER_OVERFLOW	Output buffer overflow
JPEGENC_EWL_ERROR	EWL error
JPEGENC_EWL_MEMORY_ERROR	EWL memory error
JPEGENC_HW_BUS_ERROR	Hardware bus error
JPEGENC_HW_DATA_ERROR	Hardware data error
JPEGENC_HW_TIMEOUT	Hardware timeout
JPEGENC_HW_RESERVED	Reserved for hardware
JPEGENC_SYSTEM_ERROR	System error
JPEGENC_INSTANCE_ERROR	Invalid instance
JPEGENC_HW_RESET	Hardware reset
JPEGENC_HW_SLICEINFO_TIMEOUT	sliceinfo poll timeout

Enumerator

JPEGENC_HW_UFBC_ERROR	ufbc error
-----------------------	------------

6.2.2.2 jpegEncFrameType

enum jpegEncFrameType

Picture frame type for initialization.

Enumerator

JPEGENC_YUV420_PLANAR	YYYY... UUUU... VVVV...
JPEGENC_YUV420_SEMIPLANAR	YYYY... UVUVUV...
JPEGENC_YUV420_SEMIPLANAR_VU	YYYY... VUVUVU...
JPEGENC_YUV422_INTERLEAVED_YUYV	YUYVYUYV...
JPEGENC_YUV422_INTERLEAVED_UYVY	UYVYUYVY...
JPEGENC_RGB565	16-bit RGB 16bpp
JPEGENC_BGR565	16-bit RGB 16bpp
JPEGENC_RGB555	15-bit RGB 16bpp
JPEGENC_BGR555	15-bit RGB 16bpp
JPEGENC_RGB444	12-bit RGB 16bpp
JPEGENC_BGR444	12-bit RGB 16bpp
JPEGENC_RGB888	24-bit RGB 32bpp

Enumerator

JPEGENC_BGR888	24-bit RGB 32bpp
JPEGENC_RGB101010	30-bit RGB 32bpp
JPEGENC_BGR101010	30-bit RGB 32bpp
JPEGENC_YUV420_I010	10-bit YUV 24bpp
JPEGENC_YUV420_MS_P010	10-bit YUVSP 24bpp
JPEGENC_YUV420_PLANAR_10BIT_PACKED_PLANAR	10-bit YUV Planar Packed
JPEGENC_YUV420_10BIT_PACKED_Y0L2	YUY YVY Packed.
JPEGENC_YUV420_PLANAR_8BIT_TILE_32_32	8-bit 32x32 tile
JPEGENC_YUV420_PLANAR_8BIT_TILE_16_16_PACKED_4	8-bit 16x16 tile packed 4
JPEGENC_YUV420_SEMIPLANAR_8BIT_TILE_4_4	YYYY... UVUVUV...
JPEGENC_YUV420_SEMIPLANAR_VU_8BIT_TILE_4_4	YYYY... VUVUVU...
JPEGENC_YUV420_PLANAR_10BIT_P010_TILE_4_4	YYYY... UVUV...
JPEGENC_YUV420_SEMIPLANAR_101010	YYYY... UVUV...
JPEGENC_YUV422SP_888	YYYY... UVUVUV...
JPEGENC_YUV420_UV_8BIT_TILE_128_2	YYYY... UVUV...
JPEGENC_YUV420_8BIT_TILE_8_8	8-bit 8x8 tile
JPEGENC_YUV420_10BIT_TILE_8_8	10-bit 8x8 tile
JPEGENC_YVU420_PLANAR	10-bit 8x8 tile
JPEGENC_YUV420_FBC64	8-bit 64x2 tile

Enumerator

JPEGENC_YUV420_UV_10BIT_TILE_128_2	10-bit 128x2 tile
JPEGENC_RGB888_24BIT	24-bit RGB 24bpp (MSB) BGR (LSB)
JPEGENC_BGR888_24BIT	24-bit BGR 24bpp (MSB) RGB (LSB)
JPEGENC_RBG888_24BIT	24-bit RBG 24bpp (MSB) GBR (LSB)
JPEGENC_GBR888_24BIT	24-bit GBR 24bpp (MSB) RBG (LSB)
JPEGENC_BRG888_24BIT	24-bit BRG 24bpp (MSB) GRB (LSB)
JPEGENC_GRB888_24BIT	24-bit GRB 24bpp (MSB) BRG (LSB)
JPEGENC_YVU422SP_888	YUV422 semi-planar, YYYY... VUVUVU...
JPEGENC_RGBX8888	YYYY... UUUU... VVVV... X8Y8U8V8... X2Y10U10V10... 24-bit RGB 32bpp
JPEGENC_BGRX8888	24-bit RGB 32bpp
JPEGENC_RGBX1010102	30-bit RGB 32bpp
JPEGENC_BGRX1010102	30-bit RGB 32bpp
JPEGENC_YVU420_PLANAR_10BIT_P010	YYYY... VUVUVU...
JPEGENC_Y8b	8-bit YYYY...
JPEGENC_Y10bWL	10-bit YYYY...
JPEGENC_Y10bWH	10-bit YYYY...
JPEGENC_YUV420SP10b	10-bit YUV420SP10b YYYY...UVUVUV... YVYUYVYU...
JPEGENC_YUV422_INTERLEAVED_YVYU	YVYUYVYU...
JPEGENC_YUV422_INTERLEAVED_VYUY	VYUYVYUY...

6.2.2.3 JpegEncPictureScanType

enum JpegEncPictureScanType

Specifies picture scan types for pre-processing.

This enumeration is valid only if the value of `EWLHwConfig_t.superTileXSupport` is 0.

Enumerator

JPEG_RASTER_SCAN	raster scan
JPEG_SUPERTILEX_SCAN	supertileX scan

6.2.2.4 JpegEncPictureRotation

```
enum JpegEncPictureRotation
```

Specifies the mode of rotation for initialization.

Enumerator

JPEGENC_ROTATE_90R	90-degree rotation clockwise.
JPEGENC_ROTATE_90L	90-degree rotation counter-clockwise.
JPEGENC_ROTATE_180	180-degree rotation.

6.2.2.5 JpegEncColorConversionType

```
enum JpegEncColorConversionType
```

Specifies the mode of RGB-to-YUV color space conversion.

Enumerator

JPEGENC_RGBTOYUV_BT601	Conversion to YUV values without any range change according to Rec. ITU-R BT.601.
JPEGENC_RGBTOYUV_BT709	Conversion to YUV values without any range change according to Rec. ITU-R BT.709.
JPEGENC_RGBTOYUV_USER_DEFINED	User-defined color space conversion.

Enumerator

JPEGENC_RGBTOYUV_BT2020	Conversion according to Rec. ITU-R BT.2020.
JPEGENC_RGBFULL_TO_YUVLIMIT_BT601	Conversion from full range RGB to limited range YUV according to Rec. ITU-R BT.601.
JPEGENC_RGBTOYUV_BT601_FULL_RANGE	(obsolete) Same as JPEGENC_RGBFULL_TO_YUVLIMIT_BT601.
JPEGENC_RGBTOYUV_BT601_LIMITED_RANGE	(Just for test) Conversion from RGB in range 0~219 to limited range YUV according to Rec. ITU-R BT.601. Conversion from full range RGB to limited range YUV according to Rec. ITU-R BT.709.
JPEGENC_RGBTOYUV_BT709_FULL_RANGE	(obsolete) Same as JPEGENC_RGBFULL_TO_YUVLIMIT_BT709.

6.2.2.6 JpegEncCodingType

enum `JpegEncCodingType`

Specifies the encoding type.

Enumerator

JPEGENC_WHOLE_FRAME	The frame is stored as a whole in the linear memory.
JPEGENC_SLICED_FRAME	The frame is sliced according to restart intervals, each with an input address.

6.2.2.7 JpegEncCodingMode

enum `JpegEncCodingMode`

Specifies the encoding mode.

Enumerator

JPEGENC_420_MODE	YUV 420 mode.
JPEGENC_422_MODE	YUV 422 mode (pre-processing only supports input formats JPEGENC_YUV422SP_888 and JPEGENC_YVU422SP_888).
JPEGENC_MONO_MODE	Monochrome mode.

6.2.2.8 JpegEncAppUnitsType

enum JpegEncAppUnitsType

Specifies the unit type of X-coordinate and Y-coordinate.

Enumerator

JPEGENC_NO_UNITS	No unit, X and Y specify the pixel aspect ratio.
JPEGENC_DOTS_PER_INCH	Dots per inch.
JPEGENC_DOTS_PER_CM	Dots per centimetre.

6.2.2.9 JpegEncTableMarkerType

enum JpegEncTableMarkerType

Specifies the type of marker for tables DHT and DQT.

Enumerator

JPEGENC_SINGLE_MARKER	Luma and chroma tables are written behind one marker.
JPEGENC_MULTI_MARKER	Luma and chroma tables are written behind one marker per component.

6.2.2.10 JpegEncThumbFormat

enum JpegEncThumbFormat

Specifies the encoding format of thumbnail.

Enumerator

JPEGENC_THUMB_JPEG	Thumbnail encoded using JPEG format.
JPEGENC_THUMB_PALETTE_RGB8	Thumbnail stored using one byte per pixel.
JPEGENC_THUMB_RGB24	Thumbnail stored using three bytes per pixel.

6.2.2.11 JpegEncRcMode

enum JpegEncRcMode

Specifies the RC mode.

Enumerator

JPEGENC_SINGLEFRAME	(Obsolete) Single frame mode.
JPEGENC_CBR	CBR RC mode.
JPEGENC_VBR	VBR RC mode.

6.2.2.12 JpegEncParamType

enum JpegEncParamType

Specify parameters type to update or retrieve

Enumerator

JPEGENC_PARAM_SIZE	parameters for size of encoded picture (TBD)
JPEGENC_PARAM_RC	parameters for rate control (TBD)
JPEGENC_PARAM_QUALITY	parameters for quality
JPEGENC_PRRAM_ROIMAP	parameters for ROI Map (TBD)
JPEGENC_PARAM_MAX	number for parameter type

6.2.3 Functions

6.2.3.1 JpegEncInit()

```
JpegEncRet JpegEncInit (
    const JpegEncCfg * pEncCfg,
    JpegEncInst * instAddr,
    void * ctx )
```

Initializes the encoder and returns an instance. The configuration contains stream parameters that cannot be changed during encoding.

Parameters

in	<i>config</i>	A pointer to the <code>JpegEncCfg</code> structure that contains initial configuration parameters of the encoder. These parameters cannot be changed during encoding.
in, out	<i>instAddr</i>	A pointer to a space where the new instance pointer of the encoder is stored.
in	<i>ctx</i>	A pointer to the context which is initialized in application and may be used by EWL. For example, when the device is open before initialization, the ctx can be used to transfer this data. If this parameter is not used by the application, its value can be NULL.

Returns

JpegEncRet

6.2.3.2 JpegEncRelease()

```
JpegEncRet JpegEncRelease (  
    JpegEncInst inst )
```

Releases an encoder instance.

Parameters

in	<i>inst</i>	Encoder instance
----	-------------	------------------

6.2.3.3 JpegEncSetParams()

```
JpegEncRet JpegEncSetParams (  
    JpegEncInst inst,  
    JpegEncParamType type,  
    void * param )
```

Set parameters used for encoding.

Parameters

in	<i>inst</i>	Encoder instance
in	<i>type</i>	of parameters to set
in	<i>parameters</i>	for corresponding type.

Returns

JpegEncRet

6.2.3.4 JpegEncGetParams()

```
JpegEncRet JpegEncGetParams (
    JpegEncInst inst,
    JpegEncParamType type,
    void * param )
```

Get parameters used for encoding.

Parameters

in	<i>inst</i>	Encoder instance
in	<i>type</i>	of parameters to set
out	<i>parameters</i>	for corresponding type.

Returns

JpegEncRet

6.2.3.5 JpegEncSetPictureSize()

```
JpegEncRet JpegEncSetPictureSize (
    JpegEncInst inst,
    const JpegEncCfg * pEncCfg )
```

Specifies the encoding configuration.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>pEncCfg</i>	A pointer to the current encoding configuration.

Returns

JpegEncRet

6.2.3.6 JpegEncSetThumbnail()

```
JpegEncRet JpegEncSetThumbnail (
    JpegEncInst inst,
    const JpegEncThumb * JpegThumb )
```

Specifies the thumbnail of a JPEG picture.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>JpegThumb</i>	Thumbnail information.

Returns

JpegEncRet

6.2.3.7 JpegEncEncode()

```
JpegEncRet JpegEncEncode (
    JpegEncInst inst,
    const JpegEncIn * pEncIn,
    JpegEncOut * pEncOut )
```

Generates JPEG JFIF output based on the configuration.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>pEncIn</i>	A pointer to the input and output buffer, as well as control information.
out	<i>pEncOut</i>	A pointer to the encoding result.

Returns

JpegEncRet

6.2.3.8 JpegEncEncodeRun()

```
JpegEncRet JpegEncEncodeRun (  
    JpegEncInst inst,  
    const JpegEncIn * pEncIn,  
    JpegEncOut * pEncOut )
```

Generates software markers and enables the hardware to encode one frame or one interval.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>pEncIn</i>	A pointer to input and output buffer, as well as control information.
out	<i>pEncOut</i>	A pointer to the encoding result.

Returns

JpegEncRet

6.2.3.9 JpegEncEncodeWait()

```
JpegEncRet JpegEncEncodeWait (  
    JpegEncInst inst,  
    JpegEncOut * pEncOut )
```

Waits for the completion of one encoding task.

Parameters

in	<i>inst</i>	Encoder instance.
out	<i>pEncOut</i>	A pointer to the storage space for receiving the encoding result.

Returns

JpegEncRet

6.3 Helper Functions

Typedef

- typedef VceApiVersion **JpegEncApiVersion**
Indicates API version information.
- typedef VceBuildInfo **JpegEncBuild**
Indicates build information.

6.3.1 Functions

6.3.1.1 JpegEncGetApiVersion()

```
JpegEncApiVersion JpegEncGetApiVersion (  
    void )
```

Obtains function version information.

Returns

Function version information.

6.3.1.2 JpegEncGetBuild()

```
JpegEncBuild JpegEncGetBuild (  
    u32 id,  
    const void * ctx )
```

Obtains build information about the specified core.

Parameters

in	<i>id</i>	Core ID.
in	<i>ctx</i>	Context used to initialize the instance.

Returns

Build information.

6.3.1.3 JpegEncCoreHasFeatures()

```
int JpegEncCoreHasFeatures (
    const EWLHwConfig_t * feature,
    EWLHwConfig_t * cfg )
```

Specifies whether a core supports a specified feature.

Parameters

in	<i>feature</i>	Configuration of the core.
in	<i>cfg</i>	Required configuration.

Returns

- 1: Features indicated in *cfg* are supported.
- 0: Not supported.

6.3.1.4 JpegEncGetBitsPerPixel()

```
u32 JpegEncGetBitsPerPixel (
    JpegEncFrameType type )
```

Obtains the bit depth of an input format.

Parameters

in	<i>type</i>	Picture format.
----	-------------	-----------------

Returns

Bit depth (bits per pixel) of the specified format.

6.3.1.5 JpegEncGetAlignedStride()

```
u32 JpegEncGetAlignedStride (
    int width,
    i32 input_format,
    u32 * luma_stride,
    u32 * chroma_stride,
    u32 input_alignment,
    u32 scan_type )
```

Obtains the stride in bytes based on the specified alignment and format.

Parameters

in	<i>width</i>	Picture width in pixels.
in	<i>input_format</i>	Picture format.
out	<i>luma_stride</i>	A pointer to fill the required luma stride in bytes.
out	<i>chroma_stride</i>	A pointer to fill the required chroma stride in bytes.
in	<i>input_alignment</i>	User-specified alignment requirement in bytes. If there is no requirement on the alignment, this parameter can be set to zero.

Returns

Bytes of one pixel.

6.3.1.6 JpegGetLumaSize()

```
JpegEncRet JpegGetLumaSize (
    JpegEncInst inst,
```

```

u64 * lumaSize,
u64 * dec400LumaTableSize )

```

Obtains the size information of the luma input buffer in an instance.

Parameters

in	<i>inst</i>	Encoder instance.
out	<i>lumSize</i>	A pointer to the storage space for receiving the size of the input luma buffer.
out	<i>dec400LumaTableSize</i>	A pointer to the storage space for receiving the DEC400 table size of the input luma buffer.

6.3.1.7 JpegSetLumaSize()

```

JpegEncRet JpegSetLumaSize (
    JpegEncInst inst,
    u64 lumaSize,
    u64 dec400LumaTableSize )

```

Sets the size of the luma input buffer in an instance.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>lumSize</i>	The size of the corresponding input luma buffer.
out	<i>dec400LumaTableSize</i>	The DEC400 table size of the corresponding input luma buffer.

6.3.1.8 JpegGetChromaSize()

```

JpegEncRet JpegGetChromaSize (
    JpegEncInst inst,

```

```

u64 * chromaSize,
u64 * dec400ChrTableSize )

```

Obtains the size of the chroma input buffer in an instance.

Parameters

in	<i>inst</i>	Encoder instance.
out	<i>chromaSize</i>	A pointer to the storage space for receiving the size of the corresponding input chroma buffer.
out	<i>dec400ChrTableSize</i>	A pointer to the storage space for receiving the DEC400 table size of the corresponding input chroma buffer.

6.3.1.9 JpegSetChromaSize()

```

JpegEncRet JpegSetChromaSize (
    JpegEncInst inst,
    u64 chromaSize,
    u64 dec400ChrTableSize )

```

Sets the size of the input buffer in an instance.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>chromaSize</i>	A pointer to the storage space for receiving the size of the corresponding input chroma buffer.
in	<i>dec400ChrTableSize</i>	A pointer to the storage space for receiving the DEC400 table size of the corresponding input chroma buffer.

6.3.1.10 JpegGetQpHdr()

```

int JpegGetQpHdr (
    JpegEncInst inst )

```

Obtains the QP used for encoding the current frame. The QP identifies picture encoding quality.

Parameters

in	<i>inst</i>	Encoder instance.
----	-------------	-------------------

Returns

0 to 51, with 0 indicating the best quality and 51 the poorest quality.

6.3.1.11 JpegEncGetEwl()

```
const void* JpegEncGetEwl (  
    JpegEncInst inst )
```

Obtains the EWL instance, which is used to allocate linear buffers.

Parameters

in	<i>inst</i>	Encoder instance.
----	-------------	-------------------

Returns

EWL instance internally initialized.

6.3.1.12 JpegEncGetOverlaySlice()

```
void JpegEncGetOverlaySlice (  
    JpegEncInst inst,  
    JpegEncIn * pEncIn,  
    i32 restartInterval,  
    i32 partialCoding,  
    i32 slice,
```

```
i32 sliceRows,  
u32 yOffset )
```

Obtains OSD slice information.

Parameters

in	<i>inst</i>	Encoder instance
in	<i>pEncln</i>	A pointer to the input and output buffer, as well as control information.
in	<i>restartInterval</i>	MCU rows to be encoded as one interval except for the last one.
in	<i>partialCoding</i>	Enabling status of partial encoding.
in	<i>slice</i>	Interval index of the current partial encoding.
in	<i>sliceRows</i>	MCU rows for the current interval.

6.3.1.13 JpegEncGetPerformance()

```
u32 JpegEncGetPerformance (  
    JpegEncInst inst )
```

Obtains encoding performance.

Parameters

in	<i>inst</i>	Encoder instance.
----	-------------	-------------------

Returns

Cycles used to encode one frame or one interval.

6.3.1.14 JpegGetSbiSupport()

```
u32 JpegGetSbiSupport (  
    JpegEncInst inst )
```

Specifies whether JPEG SBI is supported.

Parameters

in	inst	Encoder instance.
----	------	-------------------

Returns

- 1: Supported.
- 0: Not supported.

6.4 Low-Latency Control API

6.4.1 Functions

6.4.1.1 JpegEncGetEncodedMbLines()

```
u32 JpegEncGetEncodedMbLines (  
    JpegEncInst inst )
```

Obtains the number of the encoded input MCU rows of the current picture.

Parameters

in	inst	Encoder instance.
----	------	-------------------

Returns

The number of encoded MCU rows.

6.4.1.2 JpegEncSetInputMBLines()

```
JpegEncRet JpegEncSetInputMBLines (  
    JpegEncInst inst,  
    u32 lines )
```

Specifies the number of input buffer MCU rows of the current picture to be processed by the encoder.

Parameters

in	<i>inst</i>	Encoder instance.
----	-------------	-------------------

Returns

JpegEncRet

6.5 MJPEG Bit Rate Control API

6.5.1 Functions

6.5.1.1 JpegEncSetRateCtrl()

```
JpegEncRet JpegEncSetRateCtrl (  
    JpegEncInst inst,  
    const JpegEncRateCtrl * pRateCtrl )
```

Obtains rate control configuration.

Parameters

in	<i>inst</i>	Encoder instance.
in	<i>pRateCtrl</i>	A pointer to the storage space for receiving the rate control configuration.

Returns

JpegEncRet

6.5.1.2 JpegEncGetRateCtrl()

```
JpegEncRet JpegEncGetRateCtrl (
    JpegEncInst inst,
    JpegEncRateCtrl * pRateCtrl )
```

Sets rate control configuration.

Parameters

in	<i>inst</i>	Encoder instance.
out	<i>pRateCtrl</i>	A pointer to the storage space for receiving the rate control configuration.

Returns

JpegEncRet

6.6 Log System API

Marco Definition

- #define **APITRACEERR**(fmt, ...)

The function that obtains error logs from the encoder API module.
- #define **APITRACEWRN**(fmt, ...)

The function that obtains warning logs from the encoder API module.
- #define **APITRACE**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_API**, fmt, ##__VA_ARGS__)

The function that obtains information logs from the encoder API module.
This function is used for tracing API calls.
- #define **APITRACEPARAM_X**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_API**, fmt, ##__VA_ARGS__)

The function that obtains information logs from the encoder API module.
This function is used for tracing API pointer-type parameters.
- #define **APITRACEPARAM**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_API**, fmt, ##__VA_ARGS__)

The function that obtains information logs from the encoder API module.
This function is used for tracing API integer-type parameters.

- **#define APITRACE_ERR**(inst, fmt, ...)
The function that obtains error logs from the encoder API module, with the instance specified.
- **#define APITRACE_INFO**(inst, fmt, ...) **VCEncTraceMsg**(inst, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_API**, fmt, **##_VA_ARGS__**)
The function that obtains information logs from the encoder API module, with the instance specified.
- **#define APITRACE_PARAM**(inst, fmt, ...) **VCEncTraceMsg**(inst, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_API**, fmt, **##_VA_ARGS__**)
The function that obtains information logs from the encoder API module, with the instance specified.
This function is used for tracing API integer-type parameters.
- **#define PERFTRACE**(inst, fmt, ...) **VCEncTraceMsg**(inst, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_PERF**, fmt, **##_VA_ARGS__**)
The function that obtains information logs for tracing performance.
- **#define PTRACE_E**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_ERROR**, **VCENC_LOG_TRACE_EWL**, fmt, **##_VA_ARGS__**)
The function that obtains error logs from the EWL module.
This function is used by EWL for hardware implementation.
- **#define PTRACE_I**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_EWL**, fmt, **##_VA_ARGS__**)
The function that obtains information logs from the EWL module.
This function is used by EWL for hardware implementation.
- **#define PTRACE**(...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_EWL**, **##_VA_ARGS__**)
The function that obtains information logs from the EWL module.
- **#define EWLTRACE_I**(inst, fmt, ...) **VCEncTraceMsg**(inst, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_EWL**, fmt, **##_VA_ARGS__**)
The function that obtains information logs from the EWL module.
This function is used by EWL for C-Model implementation.
- **#define EWLTRACE_E**(inst, fmt, ...) **VCEncTraceMsg**(inst, **VCENC_LOG_ERROR**, **VCENC_LOG_TRACE_EWL**, fmt, **##_VA_ARGS__**)
The function that obtains error logs from the EWL module.
This function is used by EWL for C-Model implementation.
- **#define MEM_LOG_I**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_MEM**, fmt, **##_VA_ARGS__**)
The function that obtains information logs for tracing memory usage.
- **#define CMLTRACE**(...)
The function that obtains information logs for tracing command lines used to set up the test bench.
- **#define RC_LOG_I**(fmt, ...) **VCEncTraceMsg**(NULL, **VCENC_LOG_INFO**, **VCENC_LOG_TRACE_RC**, fmt, **##_VA_ARGS__**)
The function that obtains information logs for tracing the rate control process.

- **#define RC_LOG_E(fmt, ...)**

The function that obtains error logs for tracing the rate control process.

- **#define RC_LOG_D(fmt, ...) VCEncTraceMsg(NULL, VCENC_LOG_DEBUG, VCENC_LOG_TRACE_RC, fmt, ##__VA_ARGS__)**

The function that obtains debugging logs for tracing the rate control process.

6.6.1 Marco Definition

6.6.1.1 APITRACEERR

```
#define APITRACEERR(  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_ERROR, VCENC_LOG_TRACE_API, "[%s:%d]" fmt, \  
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

6.6.1.2 APITRACEWRN

```
#define APITRACEWRN(  
    fmt,  
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_WARN, VCENC_LOG_TRACE_API, \  
    "[%s:%d]Warning: " fmt, __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

6.6.1.3 APITRACE_ERR

```
#define APITRACE_ERR(
    inst,
    fmt,
    ... )
```

Value:

```
VCEncTraceMsg(inst, VCENC_LOG_ERROR, VCENC_LOG_TRACE_API, "[%s:%d]" fmt, \
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

6.6.1.4 CMLTRACE

```
#define CMLTRACE(
    ... )
```

Value:

```
do {
    VCEncTraceMsg(NULL, VCENC_LOG_INFO, VCENC_LOG_TRACE_CML, ##__VA_ARGS__); \
} while (0)
```

6.6.1.5 RC_LOG_E

```
#define RC_LOG_E(
    fmt,
    ... )
```

Value:

```
VCEncTraceMsg(NULL, VCENC_LOG_ERROR, VCENC_LOG_TRACE_RC, "[%s:%d]" fmt, \
    __FUNCTION__, __LINE__, ##__VA_ARGS__)
```

6.6.2 Enumeration Type

6.6.2.1 VCEncLogEnvControl

```
enum VCEncLogEnvControl
```

Defines log control information types obtained from the environment.

Enumerator

VCENC_LOG_OUTPUT	Information for the environment to control output files.
VCENC_LOG_LEVEL	Information for the environment to control the output level.
VCENC_LOG_TRACE	Information for the environment to control trace information for prompt and debugging.
VCENC_LOG_CHECK	Information for the environment to control test-only information check.

6.6.2.2 VCEncLogOutput

```
enum VCEncLogOutput
```

Defines log output modes.

Enumerator

LOG_STDOUT	Outputs all logs to stdout.
LOG_ONE_FILE	Outputs all logs to the same file. The log file is <code>vcenc_trace_p\${pid}.log</code> where <code>\${pid}</code> is the process ID.
LOG_BY_THREAD	Outputs logs of each thread to a separate log file. The log files are named <code>vcenc_trace_p\${pid}t\${tid}.log</code> where <code>\${pid}</code> is the process ID and <code>\${tid}</code> is the thread ID.
LOG_STDERR	Outputs all logs to stderr.

6.6.2.3 VCEncLogFileType

```
enum VCEncLogFileType
```

Defines log types.

Enumerator

STREAM_TRACE_FILE	Trace log, used to debug or analyze the status of the encoder behavior.
STREAM_CHECK_FILE	(Reserved) Check log, which is used in tests to check whether a feature is enabled and properly configured.

6.6.2.4 VCEncLogTraceType

enum VCEncLogTraceType

Defines trace log types.

Enumerator

VCENC_LOG_TRACE_API	Encoder API call log.
VCENC_LOG_TRACE_REGS	Register configuration log.
VCENC_LOG_TRACE_EWL	EWL API call log.
VCENC_LOG_TRACE_MEM	Memory usage log.
VCENC_LOG_TRACE_RC	Rate control status log.
VCENC_LOG_TRACE_CML	Command line log.
VCENC_LOG_TRACE_PERF	Performance log.

6.6.2.5 VCEncLogCheckType

enum VCEncLogCheckType

(Reserved) Defines check log types.

Enumerator

VCENC_LOG_CHECK_RECON	Reconstructed YUV data.
VCENC_LOG_CHECK_QUALITY	PSNR/SSIM for each frame.

Enumerator

VCENC_LOG_CHECK_VBV	VBV information for rate control checking.
VCENC_LOG_CHECK_RC	Rate control information for rate control profiling.
VCENC_LOG_CHECK_FEATURE	Feature information for coverage checking.

6.6.2.6 VCEncLogLevel

enum `VCEncLogLevel`

Defines output log levels.

Enumerator

VCENC_LOG_QUIET	No logs output.
VCENC_LOG_FATAL	The fatal level.
VCENC_LOG_ERROR	The error level.
VCENC_LOG_WARN	The warning level.
VCENC_LOG_INFO	The information level.
VCENC_LOG_DEBUG	The debugging level.
VCENC_LOG_ALL	All logs output.

6.6.3 Functions**6.6.3.1 VCEncLogInit()**

```
int VCEncLogInit (
    unsigned int out_dir,
    unsigned int out_level,
    unsigned int trace_map,
    unsigned int check_map )
```

Initializes the log system.

Parameter settings are prioritized in ascending order as follows: default values < environment settings < command line settings.

Parameters

in	<i>out_dir</i>	The log output mode.
in	<i>out_level</i>	The lowest log level to be output. Logs of the specified level and higher are output.
in	<i>trace_map</i>	A bitmap where each bit controls whether to output each type of trace logs. 0: do not output. 1: output. For the mappings between bit indexes and trace log types, see Section <i>VCEncLogTraceType</i> .
in	<i>check_map</i>	(Reserved) A bitmap where each bit controls whether to output each type of check logs. 0: do not output. 1: output. For the mappings between bit indexes and check log types, see Section <i>VCEncLogCheckType</i> .

6.6.3.2 VCEncTraceMsg()

```
void VCEncTraceMsg (
    const void * inst,
    VCEncLogLevel level,
    unsigned int log_trace_mask,
    const char * fmt,
    ... )
```

Prints trace logs of a given type and levels.

Parameters

in	<i>inst</i>	The instance for which trace logs are to be printed.
----	-------------	--

Parameters

in	<i>level</i>	The lowest log level to be output. Logs of the specified level and higher are output.
in	<i>log_trace_mask</i>	A bitmap where each bit controls whether to print each type of trace logs. 0: do not print. 1: print. For the mappings between bit indexes and trace log types, see Section <i>VCEncLogTraceType</i> .
in	<i>fmt</i>	A string in the printf format.

6.6.3.3 VCEncCheckMsg()

```
void VCEncCheckMsg (
    const void * inst,
    VCEncLogLevel level,
    unsigned int log_check_mask,
    const char * fmt,
    ... )
```

(Reserved) Prints check logs of a given type and levels.

6.6.3.4 VCEncLogDestory()

```
int VCEncLogDestory (
    void )
```

Releases resources allocated to the log system.

6.6.3.5 EncTraceRegs()

```
void EncTraceRegs (
    const void * ewl,
    unsigned int readWriteFlag,
    unsigned int mbNum,
    unsigned int * regs )
```

(Reserved) Outputs register information.

6.6.3.6 VCEncLogGetEnvSetting()

```
void VCEncLogGetEnvSetting (
    VCEncLogSetting * env_log )
```

Queries the parameters for controlling the log system.

Parameters

in	<i>env_log</i>	The log settings read from the environment or specified during initialization.
----	----------------	--

7 Data Structure Documentation

7.1 jpeg_ufbc_afbc_param

Contains parameters when the core of UFBC is AFBC.

Data Fields

- **u32 yuvTrans**
Whether to enable RGB-to-YUV conversion in UFBC.
0: disable.
1: enable.
- **u32 blockType**
The superblock type of UFBC.
0: 32x8.
1: 16x16.
- **u32 blockSplit**
Whether to enable block split mode in UFBC.
0: Block split mode off.
1: Block split mode on.

7.2 jpeg_ufbc_param

Contains the UFBC parameters.

Data Fields

- jpeg_ufbcMode **mode**

Whether to enable block split mode in UFBC.

JPEG_UFBC_MODE_NONE: UFBC disable.

JPEG_UFBC_MODE_AFBC_0: Use the core of AFBC version 0.(only support 32x8 superblock).

JPEG_UFBC_MODE_AFBC_1: Use the core of AFBC version 1.(Support 32x8 and 16x16 superblock).

JPEG_UFBC_MODE_DEC400: Use the core of Dec400.

•

union {

 jpeg_ufbc_afbc_param **afbc**

 jpeg_ufbc_pvric_param **pvric**

} **param**

7.3 jpeg_ufbc_pvric_param

Contains parameters when the core of UFBC is PVRIC.

Data Fields

- **u32 blockType**

Surface tile mode.

0: 8x8.

1: 16x4.

2: 32x2.

- **u32 consColorVal [4]**

constant color value.

7.4 JpegEncCfg

Indicates encoder configuration.

Data Fields

- **u32 inputWidth**
The number of pixels per line in an input picture.
- **u32 inputHeight**
The number of lines in an input picture.
- **u32 xOffset**
The horizontal offset of top-left encoded picture relative to the top-left corner of an input picture.
- **u32 yOffset**
The vertical offset of top-left encoded picture relative to the top-left corner of an input picture.
- **u32 codingWidth**
The width of an encoded picture.
- **u32 codingHeight**
The height of an encoded picture.
- **u32 restartInterval**
The restart interval in MCU rows.
- **i32 quality**
the quality factor from 1 to 100 when quality factor is -1, use `qLevel`
- **u32 qLevel**
The quantization level.
Levels 0 through 9 use the predefined quantization table.
Level 10 uses the external quantization table specified in `qTableLuma` and `qTableChroma`.
- **const u8 * qTableLuma**
Quantization table for luminance, which is used when the quantization level is 10.
- **const u8 * qTableChroma**
Quantization table for chrominance, which is used when the quantization level is 10.
- **u8 filter [128]**
Two 8x8 filters defined for luma and chroma non-ROI regions.
- **u32 enableRoimap**
Status of the ROI map feature.
- **u32 nonRoiLevel**
Non-ROI filter level (0 to 9).
- **JpegEncFrameType frameType**
Input frame in YUV or RGB format.
- **JpegEncColorConversion colorConversion**
Whether to enable RGB-to-YUV conversion.
0: Disable.
1: Enable.
- **JpegEncPictureRotation rotation**

Picture rotation mode.

- **u32 mirror**

Whether to enable mirror.

0: Disable.

1: Enable.

- **JpegEncCodingType codingType**

Encoding type, which can be either whole frame or restart interval.

- **JpegEncCodingMode codingMode**

Encoding mode, which can be YUV420, monochrome, or YUV422 coding (pre-processing only supports one special YUV422 format).

- **JpegEncAppUnitsType unitsType**

Unit for X and Y density in APP0.

- **JpegEncPictureScanType scanType**

Specifies picture scan types for pre-processing.

- **JpegEncTableMarkerType markerType**

Table marker type.

- **u32 xDensity**

The horizontal pixel density.

- **u32 yDensity**

The vertical pixel density.

- **u32 comLength**

The comment header length.

- **const u8 * pCom**

The comment header pointer.

- **u32 inputLineBufEn**

Whether to enable input picture control signals.

0: Disable.

1: Enable.

- **u32 inputLineBufLoopBackEn**

Whether to enable input buffer loopback.

0: Disable.

1: Enable.

- **u32 inputLineBufDepth**

Input buffer depth in MCU rows.

- **u32 inputLineBufHwModeEn**

Whether to enable hardware handshake.

0: Disable.

1: Enable.

- **u32 amountPerLoopBack**

Synchronous handshake amount for every loopback.

- **EnclInputLineBufCallBackFunc inputLineBufCbFunc**
Callback function.
- **void * inputLineBufCbData**
Callback function data.
- **u32 hashType**
The hash type for verifying stream correctness.
0: Disabled.
1: CRC32
2: Checksum.
- **i32 constChromaEn**
Whether the constant chroma control is enabled.
If enabled, chroma is encoded as constant values.
0: Disabled.
1: Enabled.
- **u32 constCb**
The constant value for the U component, when constant chroma control is enabled.
- **u32 constCr**
The constant value for the V component, when constant chroma control is enabled.
- **u32 losslessEn**
Whether to enable lossless mode.
If enabled, JPEG lossless mode (SOF3) is used.
0: Disable.
1: Enable.
- **u32 predictMode**
The predictor mode used for JPEG lossless processing. The value of the nominator of frame rate ranges from 0 to 7.
- **u32 ptransValue**
A point transform value used for lossless processing.
- **u32 targetBitPerSecond**
Target bit per second.
- **u32 frameRateNum**
The numerator of the frame rate. Value range [1, 1048575].
- **u32 frameRateDenom**
The denominator of the frame rate. Value range [1, frameRateNum].
The frame rate is the result of frameRateNum divided by frameRateDenom in frames per second.
- **u32 qpmin**
The minimum QP allowed during rate control for motion JPEG.
- **u32 qpmax**
The maximum QP allowed during rate control for motion JPEG.
- **i32 fixedQP**
Fixed QP without rate control.

- **JpegEncRcMode rcMode**
Rate control mode.
- **i32 picQpDeltaMin**
Obsolete.
- **i32 picQpDeltaMax**
Obsolete.
- **u32 exp_of_input_alignment**
Stride used for input picture width.
- **u32 slice_idx**
Slice node index.
- **u32 streamMultiSegmentMode**
Stream multi-segment control mode.
 - 0: Segment disabled.
 - 1: (Reserved) Multi-segment with loopback and no synchronous control.
 - 2: (Reserved) Multi-segment with loopback and software synchronous control.
 - 3: Multi-segment without loopback and no synchronous control.
- **u32 streamMultiSegmentAmount**
The total number of stream multi-segment buffers.
The number must be more than one.
When loopback is enabled, streams should be written from the start address after the last segment is filled.
- **u32 streamMultiSegmentSize**
Stream segment buffer size.
Callback is triggered when one segment is filled.
- **EncStreamMultiSegCallBackFunc streamMultiSegCbFunc**
Callback function is triggered when one stream segment is filled.
- **void * streamMultiSegCbData**
Callback data context for stream multi-segment callback function.
- **u32 burstMaxLength**
The maximum AXI burst length.
- **u32 AXIAlignment**
AXI alignment control.
 - bit [31:28] AXI_burst_align_wr_common.
 - bit [27:24] AXI_burst_align_wr_stream.
 - bit [23:20] AXI_burst_align_wr_chroma_ref.
 - bit [19:16] AXI_burst_align_wr_luma_ref.
 - bit [15:12] AXI_burst_align_rd_common.
 - bit [11:8] AXI_burst_align_rd_prp.
 - bit [7:4] AXI_burst_align_rd_ch_ref_prefetch.
 - bit [3:0] AXI_burst_align_rd_lu_ref_prefetch.
- **u32 irqTypeMask**

Interrupt request type mask.

swreg1 bit 8 irq_type_sw_reset_mask.

swreg1 bit 7 irq_type_fuse_error_mask.

swreg1 bit 6 irq_type_buffer_full_mask.

swreg1 bit 5 irq_type_bus_error_mask.

swreg1 bit 4 irq_type_timeout_mask.

swreg1 bit 3 irq_type_strm_segment_mask.

swreg1 bit 2 irq_type_line_buffer_mask.

swreg1 bit 1 irq_type_slice_rdy_mask.

swreg1 bit 0 irq_type_frame_rdy_mask.

- **u32 olFrameNum** [MAX_OVERLAY_NUM]

The number of OSD input frames recorded to support the roll back feature.

- **u32 olEnable** [MAX_OVERLAY_NUM]

Whether the control of each OSD area is enabled.

0: Disabled.

1: Enabled.

- **u32 olFormat** [MAX_OVERLAY_NUM]

The input formats of each OSD area.

0: ARGB8888.

1: NV12.

2: Bitmap.

- **u32 olAlpha** [MAX_OVERLAY_NUM]

The global alpha value of each OSD area, not used for ARGB8888.

- **u32 olWidth** [MAX_OVERLAY_NUM]

Input source width of an OSD area.

- **u32 olCropWidth** [MAX_OVERLAY_NUM]

The width of each OSD area after cropping.

- **u32 olHeight** [MAX_OVERLAY_NUM]

The input source height of each OSD area.

- **u32 olCropHeight** [MAX_OVERLAY_NUM]

The height of an OSD area after cropping.

- **u32 olXOffset** [MAX_OVERLAY_NUM]

The top left column offset of an OSD area.

- **u32 olCropXOffset** [MAX_OVERLAY_NUM]

The horizontal offset X, which is the left edge of the cropping region.

- **u32 olYOffset** [MAX_OVERLAY_NUM]

Top left row offset of an OSD area.

- **u32 olCropYOffset** [MAX_OVERLAY_NUM]

The vertical offset Y, which is the top edge of the cropping region.

- **u32 olYStride** [MAX_OVERLAY_NUM]

The input luma stride which is stored for an OSD area.

- **u32 olUVStride** [MAX_OVERLAY_NUM]
The input chroma stride which is stored for an OSD area.
- **u32 olBitmapY** [MAX_OVERLAY_NUM]
The global luma Y value for bitmap format.
- **u32 olBitmapU** [MAX_OVERLAY_NUM]
The global chroma Cb value for bitmap format.
- **u32 olBitmapV** [MAX_OVERLAY_NUM]
The global chroma Cr value for bitmap format.
- **u32 olSuperTile** [MAX_OVERLAY_NUM]
Whether the input is in super tile format. (For ARGB8888 format only)
0: Not in super tile format.
1: X-major super tile format.
2: Y-major super tile format.
- **u32 olScaleWidth** [MAX_OVERLAY_NUM]
(Reserved) The width of an OSD area after upscaling.
- **u32 olScaleHeight** [MAX_OVERLAY_NUM]
(Reserved) The height of an OSD area after upscaling.
- **u32 osdDec400TableSize** [MAX_OVERLAY_NUM][3]
The size of the compression table buffer used by DEC400.
- **FILE * osdDec400TableFile** [MAX_OVERLAY_NUM]
- **u32 mosEnable** [MAX_MOSAIC_NUM]
Whether the Mosaic region is enabled. (Not support Monochrome encoding if enable)
0: Disabled.
1: Enabled.
- **u32 mosWidth** [MAX_MOSAIC_NUM]
The width of a mosaic region.
- **u32 mosHeight** [MAX_MOSAIC_NUM]
The height of a mosaic region.
- **u32 mosXoffset** [MAX_MOSAIC_NUM]
The top left horizontal offset of a mosaic region.
- **u32 mosYoffset** [MAX_MOSAIC_NUM]
The top left vertical offset of a mosaic region.
- **u32 mosSizeIndex**
- **u32 osdMapEnable**
- **u32 osdMapStride**
- **u32 osdMapBlockSize**
- **u32 osdMapAlpha** [MAX_OSDMAP_COLOR_NUM]
- **u32 osdMapY** [MAX_OSDMAP_COLOR_NUM]
- **u32 osdMapU** [MAX_OSDMAP_COLOR_NUM]

- **u32 osdMapV** [MAX_OSDMAP_COLOR_NUM]
- **EWLinearMem_t * osdMapMem**
- **u32 sbi_id_0**
The FLEXA SBI ID is 0.
- **u32 sbi_id_1**
The FLEXA SBI ID is 1.
- **u32 sbi_id_2**
The FLEXA SBI ID is 2.
- **u32 segmentUnitHeight**
The FLEXA SBI segment unit height.
- **u32 sramPowerdownDisable**
Whether powering down the internal SRAM automatically is enabled.
0: Enabled.
1: (Default) Disabled.
- **u32 sramPowerdownMode**
- **u32 sramPowerdownTimerDiv32**
- **u32 dumpRegister**
The debug option for dumping register.
- **jpeg_ufbc_param ufbcParam**
Contains the UFBC parameters.
- **u32 secure_mode**
Whether security mode for encoding is enabled.
0: Disabled.
1: Enabled.
- **u32 sliceinfoEn**
Whether polling slice information mode of input low latency is enabled.
0: Disabled.
1: Enabled.
- **u32 priority**
the priority of current instance
- **u32 core_mask**
the core bit mask of current instance
0x0: not specify, anycore.
0x1: sepcfiy core0.
0x2: sepcfiy core1.
- **char * enc_dev**
the device name of enc driver.
default is "vsi_vcx"
- **char * mem_dev**
the device name of memalloc driver
default is "memalloc"

- **u32 useVcmd**
*Whether to use vcmd.
This field is valid only for cmodel.*
- **u32 lowlatGatingDisable**
*lowlatency handshake auto gating disable
0: enable.
1: disable.*

7.5 JpegEncColorConversion

Configures RGB-to-YUV color space conversion. For how the following parameters are used, see Section [Color Space Conversion](#).

Data Fields

- **JpegEncColorConversionType type**
Pre-defined color conversion types. Besides the user-defined type, some standard color conversion types are defined for setting coefficients quickly.
- **u16 coeffA**
User-defined color space conversion coefficient A.
- **u16 coeffB**
User-defined color space conversion coefficient B.
- **u16 coeffC**
User-defined color space conversion coefficient C.
- **u16 coeffE**
User-defined color space conversion coefficient E.
- **u16 coeffF**
User-defined color space conversion coefficient F.
- **u16 coeffG**
User-defined color space conversion coefficient G.
- **u16 coeffH**
User-defined color space conversion coefficient H.
- **u16 LumaOffset**
User-defined color space conversion parameter LumaOffset.

7.6 JpegEncln

Configures encoding input information.

Data Fields

- **u32 frameHeader**

Whether to create a frame header.

0: No.

1: Yes.

- **ptr_t busLum**

The bus address for input luminance (Y) picture.

Planar format: luminance component.

Semi-planar format: luminance component.

Interleaved format: whole picture.

- **ptr_t busCb**

The bus address for input chrominance (U).

Planar format: U component.

Semi-planar format: both chrominance.

Interleaved format: not used.

- **ptr_t busCr**

The bus address for input chrominance (V).

Planar format: V component.

Semi-planar format: not used.

Interleaved format: not used.

- **ptr_t busRoiMap**

The bitmap buffer for ROI definition. Each bit defines one MCU.

- **u8 * filter**

Quantization table filter, when 0 is set in the corresponding bit of the ROI map buffer.

- **const u8 * pLum**

Virtual address for input luminance (Y).

- **const u8 * pCb**

Virtual address for input chrominance (U).

- **const u8 * pCr**

Virtual address for input chrominance (V).

- **ptr_t dec400TableBusLum**

The bus address of DEC400 compression table for luminance input (Y).

- **ptr_t dec400TableBusCb**

The bus address of DEC400 compression table for chrominance input (U).

- **ptr_t dec400TableBusCr**

The bus address of DEC400 compression table for chrominance input (V).

- **const u8 * dec400TablepLum**

The virtual address of DEC400 compression table for luminance input (Y).

- **const u8 * dec400TablepCb**

The virtual address of DEC400 compression table for chrominance input (U).

- **const u8 * dec400TablepCr**

The virtual address of DEC400 compression table for chrominance input (V).

- **u32 dec400Enable**

[0,7] Whether to enable DEC400.

0: global-bypass or indicate dec400 doesn't exit.

1: stream-bypass.

2: enable.

- **ptr_t sliceinfoBus**

Bus adress of sliceinfo data.

- **u32 axiFEEEnable**

The working mode of AXI_FE.

0: Disabled.

1: Normal mode.

2: Bypass.

3: Security mode.

- **u8 * pOutBuf [MAX_STRM_BUF_NUM]**

The virtual address of the output stream buffer.

- **ptr_t busOutBuf [MAX_STRM_BUF_NUM]**

The bus address of the output stream buffer.

- **u32 outBufSize [MAX_STRM_BUF_NUM]**

The output buffer size in bytes.

- **u32 lineBufWrCnt**

The number of MCU rows in the input MCU row buffer.

- **u32 initSegNum**

The number of segments where input data is stored.

- **u32 overlayEnable [MAX_OVERLAY_NUM]**

Whether to enable one overlay region.

0: Disable.

1: Enable.

- **ptr_t busOILum [MAX_OVERLAY_NUM]**

The bus address of the luma component buffer of an overlay picture.

- **ptr_t busOICb [MAX_OVERLAY_NUM]**

The bus address of the Cb component buffer of an overlay picture.

- **ptr_t busOICr [MAX_OVERLAY_NUM]**

The bus address of the Cr component buffer of an overlay picture.

- **u32 osdDec400Enable** [MAX_OVERLAY_NUM]
Whether the overlay picture of region 0 is compressed by DEC400.
0: No.
1: Bypass.
2: Yes.
- **ptr_t osdDec400TableBase** [MAX_OVERLAY_NUM][3]
The bus address of the DEC400 compression table when the overlay picture of region 0 is compressed by DEC400.
- **u32 picCnt**
(For internal use only) The number of pictures counted.
- **u32 osdMapEnable**
OSDMap enable.
- **ptr_t osdMapInputAddr**
OSD MAP buffer base address.
- **u32 fcEnable** [3]
Enable the fast clear funtion of dec400.
- **u32 clearColorLow** [3]
The clear color in 16 bpp or 32 bpp for YUV channel.
- **u32 dec400TSHdrEnable**
Whether to process dec400 tile status header buffer.

7.6.1 Field Documentation

7.6.1.1 dec400Enable

u32 dec400Enable

[8,15] DEC400 tileSize.

- 0: default.
- 1: 128byte.
- 2: 512byte.
- 3: 256byte.

[16,23] DEC400 tileMode.

- 0: default.
- 1: TILE_8x8_x.
- 2: TILE_8x8_y.

- 3: TILE_16x4.
- 4: TILE_8x4.
- 5: TILE_4x8.
- 6: RASTER_16x4.
- 7: TILE_64x4.
- 8: TILE_32x4.
- 9: RASTER_256x1.
- 10: RASTER_128x1.
- 11: RASTER_64x1.
- 12: TILE_16x8.
- 13: RASTER_32x4.
- 14: RASTER_32x1.
- 15: RASTER_16x1.
- 16: TILE_8x4_s.
- 17: TILE_16x4_s.
- 18: TILE_32x4_s.
- 19: TILE_32x8.

[25,27] DEC400 RGB format.

- 0: default.
- 1: RGB8.
- 2: RGB10.
- 3: RGB4.
- 4: RGB1555.
- 5: RGB565.

[28,28] DEC400 RGB format X or A.

- 0: ARGB.
- 1: XRGB.

[29,31] DEC400 data alignment.

- 0: default.
- 1: 32byte.
- 2: 64byte.
- 3: 1byte.
- 4: 16byte.

7.7 JpegEncOut

Indicates output information.

Data Fields

- **u32 jfifSize**
Encoded JFIF size in bytes.
- **u32 invalidBytesInBuf0Tail**
Unused bytes in buf0 tail.
- **u32 headerSize**
JPEG header size in bytes.

7.8 JpegEncQuality

Parameters to set quality of encoded picture.

Data Fields

- **i32 factor**

7.8.1 Field Documentation

7.8.1.1 factor

i32 factor

valid quality factor is from 1 to 100. when factor is -1, other method will be used to adjust the quantization table.

7.9 JpegEncRateCtrl

Indicates the JPEG rate control parameters.

Data Fields

- **u32 pictureRc**

Whether to enable bit rate control by adjusting quantization parameters. Value range [0, 1].

0: Disable.

1: Enable.

- **i32 qpHdr**

QP for the next encoded picture when rate control is enabled. Value range [-1, 51].

The value -1 indicates rate control calculates the initial QP for the first picture.

Other values indicate this QP is used as the initial QP for the first picture and rate control calculates the QP for the following pictures.

- **u32 qpMin**

Reserved.

- **u32 qpMax**

Reserved.

- **u32 bitPerSecond**

Target bit rate in the unit of bits per second. It is mandatory when RC is enabled. Value range [10000, 60000000].

- **u32 hrd**

(Reserved) Whether to enable HRD.

0: Disable.

1: Enable.

- **u32 bitrateWindow**

Bit rate control window. Value range [1, 300].

The target bit rate is recommended in input frames.

- **u32 fixedIntraQp**

Fixed QP value for all intra pictures. Value range [0, 51].

The value 0 indicates rate control calculates intra QP.

- **i32 tolMovingBitRate**

Tolerance of the maximum moving bit rate, which is the maximum limit of the instantaneous bit rate.

- **i32 monitorFrames**

Reserved.

- **i32 picQpDeltaMin**

The minimum picture QP delta. It is used to smooth quality change between pictures.

- **i32 picQpDeltaMax**

The maximum picture QP delta. It is used to smooth quality change between pictures.

- **u32 frameRateNum**

Frame rate.

- **u32 frameRateDenom**

The maximum frame rate is the result of `frameRateNum` divided by `frameRateDenom` in frames per second.

The actual frame rate is defined by time increment of encoded pictures.

7.10 JpegEncThumb

Thumbnail information.

Data Fields

- **JpegEncThumbFormat** format

Thumbnail format.

- **u16** width

Thumbnail width in pixels.

- **u16** height

Thumbnail height in pixels.

- **const void *** data

Thumbnail data.

- **u32** dataLength

Data amount in bytes.

7.11 VCEncLogSetting

Contains the log settings read from the environment or specified during initialization.

Data Fields

- **VCEncLogOutput** out_dir

The log output mode.

- **VCEncLogLevel** out_level

The lowest log level to be output.

Logs of the specified level and higher are output.

- **unsigned int** k_trace_map

A bitmap where each bit indicates whether to output each type of trace logs.

0: do not output.

1: output.

*For the mappings between bit indexes and trace log types, see Section **VCEncLogTraceType**.*

*The values are parsed from the `trace_map` parameter in **VCEncLogInit()**.*

- **unsigned int** k_check_map

(Reserved) A bitmap where each bit indicates whether to output each type of check logs.

0: do not output.

1: output.

*For the mappings between bit indexes and check log types, see Section **VCEncLogCheckType**.*

*The values are parsed from the `check_map` parameter in **VCEncLogInit()**.*

Appendix: Glossary

The following table describes the terms used in this document. For additional HEVC acronyms, see <http://www.hevcbook.de/acronyms/>.

Term	Description
1080p	high-definition resolution of 1920 x 1080 progressive video
720p	high-definition resolution of 1280 x 720 progressive video
ABR	average bit rate
AIoT	artificial intelligence of things
APB	Advanced Peripheral Bus
API	application programming interface
ARIB	Association of Radio Industries and Businesses
ASIC	application-specific integrated circuit
AV1	AOMedia Video 1
AVC	Advanced Video Coding (H.264)
AXI	Advanced eXtensible Interface
bps	bits per second
CABAC	context-adaptive binary arithmetic coding
CAVLC	content-adaptive variable-length coding
CB	coding block
CBR	constant bit rate
CIF	Common Interchange Format (352 x 288 pixels)
CIR	cyclic intra refresh
CPB	coded picture buffer
CPU	central processing unit
CQP	constant quantization parameter
CRF	constant rate factor
CSC	color space conversion
CTB	coding tree block
CTU	coding tree unit

Term	Description
CU	coding unit
CVBR	constrained variable bit rate
DDR	double data rate
DRM	digital rights management
EFBC	external frame buffer compression
EOS	end of sequence
EWL	encoder wrapper layer
FPS	frames per second
GDR	gradual decoder refresh
GOP	group of pictures
HD	high definition
HDTV	high-definition television
HEIF	High Efficiency Image File Format
HDR	high dynamic range
HEVC	High Efficiency Video Coding, a video coding standard (H.265) developed by JCT-VC
HRD	hypothetical reference decoder
IDR	Instantaneous Decoder Refresh
IPCM	intra pulse code modulation
ITU	International Telecommunication Union
ITU-T	ITU Telecommunication Standardization Sector
ITU-R	ITU Radiocommunication Sector
Kbps	kilobits per second
LCU	largest CU
LTR	long-term reference
MAD	mean absolute difference
MB	macroblock
MMU	memory management unit
MPEG-4	Moving Picture Experts Group 4
MV	motion vector
NAL	network abstraction layer
NALU	NAL unit, the smallest decodable unit of a stream
OBU	open bitstream unit
OSD	on-screen display
PAL	Phase Alternate Line (720 x 576 pixels resolution)
POC	picture order count
PPS	picture parameter set
PSNR	peak signal to noise ratio

Term	Description
PU	prediction unit
QCIF	Quarter CIF (176 x 144 pixels)
QVGA	Quarter Video Graphics Array (320 x 240 pixels)
QP	quantization parameter
RC	rate control
RDO	rate-distortion optimization
RDOQ	rate-distortion optimized quantization
REE	rich execution environment
RFC	reference frame compression
RGB	red, green, blue color space representation
ROI	region of interest
RPS	reference picture set
SAO	sample adaptive offset
SBI	segment buffer interface
SEI	supplemental enhancement information
SMPTE	Society of Motion Picture and Television Engineers
SPS	sequence parameter set
SSIM	structural similarity index measure
STR	short-term reference
sub-QCIF	Sub Quarter CIF (128 x 96 pixels resolution)
SVC-T	temporal scalable video coding
SXGA	Super Extended Graphics Array (1280 x 1024 pixels resolution)
TU	transformation unit
TV	television
VBR	variable bit rate
VBV	video buffer verifier
VGA	Video Graphics Array (640 x 480 pixels resolution)
VLC	variable-length code
VPS	video parameter set
VUI	video usability information
YCbCr	color space with one luma and two chroma components, used for digital encoding
YUV	color space with one luma and two chroma component, used for either digital or analog encoding

Document Revision History

This section describes top level differences in the versions of this document.

NOTE: This document is not necessarily updated for each patch or minor revision. The information in this document tends to be stable across a revision (nnn) series.

Document Revision	Date	**Compatible Core**	Description
1.07	2024-01-03	VC9000E	Updated the document name and template.

Document Revision	Date	**Compatible Core**	Description
1.06	2023-06-12	VC9000E	- Reconstructed the document and refined descriptions. Section <i>Encoded Picture Size</i> Updated the height and width limitations on picture size. Section <i>ROI Map</i> Added bit value description about ROI map buffer. - Added Section <i>Pre-Processing</i> and Section <i>Stream Segment for Low Latency</i>. - Moved the content in Section <i>Size Limitations</i> to Section <i>Encoded Picture Size</i>. Section <i>JpegEncRet</i> Added enumeration values. Section <i>JpegEncAppUnitsType</i> Added the JPEGENC_DOTS_PER_INCH enumeration value. Section <i>Log and Message Utility API</i> Added macro definition. Section <i>VCEncTraceMsg</i> Changed the log_trace_level parameter to log_trace_mask. Section <i>VCEncCheckMsg</i> Changed the log_check_level parameter to log_check_mask. Section <i>JpegEncCfg</i> - Added fields: efbcEnable, efbcYuvTrans, secure_mode, and sliceinfoEn. - Removed the fields FILE*olFile [MAX_OVERLAY_NUM] and FILE*osdDec400TableFile. Section <i>JpegEncln</i> Added the field picCnt. Section <i>JpegEncOut</i> Removed the field invalidBytesInBuf0Tail.

Document Revision	Date	**Compatible Core**	Description
1.05	2021-07-29	VC9000E	Added section 1 (Overview) Added Numeric Data Type.
1.04	2018-08-01	VC8000E	Section 3: added Inputs and Outputs sections. JpegEncInit function: added more fields used in the JpegEncCfg structure. JpegEncSetPictureSize function: added more fields used in the JpegEncCfg structure. JpegEncEncode: added lineBufWrCnt field to list of fields used in the JpegEncIn structure. JpegEncEncodeRun: added lineBufWrCnt to list of fields used in the JpegEncIn structure.
1.03	2018-05-29	VC8000E	Section 2.3.6: Added new members to JpegEncFrameType enumeration. Section 2.4.3: Added exp_of_input_alignment data member to JpegEncCfg structure. Section 3: removed JpegEncInst() function.
1.02	2018-03-12	VC8000E	Added Section 4.2.1. Support for multiple streams.

Document Revision	Date	**Compatible Core**	Description
1.01	2018-02-05	VC8000E	Added MJPEG to supported standards. JpegEncFrameType: added JPEGENC_BGR555. JpegEncCfg: added frameRateNum, frameRateDenom, qpmin, qpmax, fixedQP. JpegEncCfg: changed targetBitPerPic to targetBitPerSecond. JpegEncColorConversionType: corrected coefficient fields. JpegEncGetBuild: added core_id parameter. Added new functions: JpegEncCoreHasFeatures, JpegEncGetBitsPerPixel, JpegEnc_Trace, JpegEncEncodeRun, JpegEncEncodeWait. Added IDX_CHUNK structure for MJPEG. Added three new MJPEG functions: MjpegAVIchunkheader, MjpegEncodeAVIHeader, MjpegAVIidx.
1.00	2017-11-29	VC8000E	Initial Release